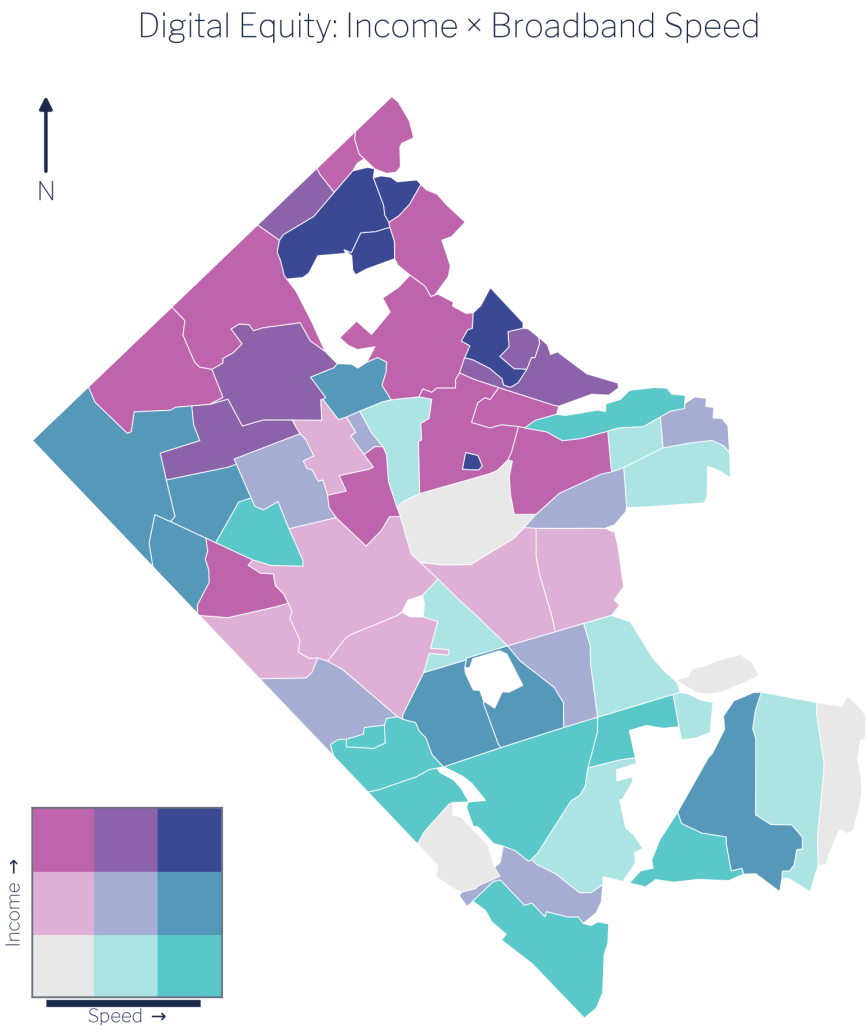
Creating Local-Level Geographic Datasets

A practical, illustrated guide for sub-county policy analysis

Aaron Schroeder

2026-06-04

Creating Local-Level Geographic Datasets



Digital equity across Arlington's 62 civic associations: median household income against broadband download speed.

1 The Sub-County Data Gap

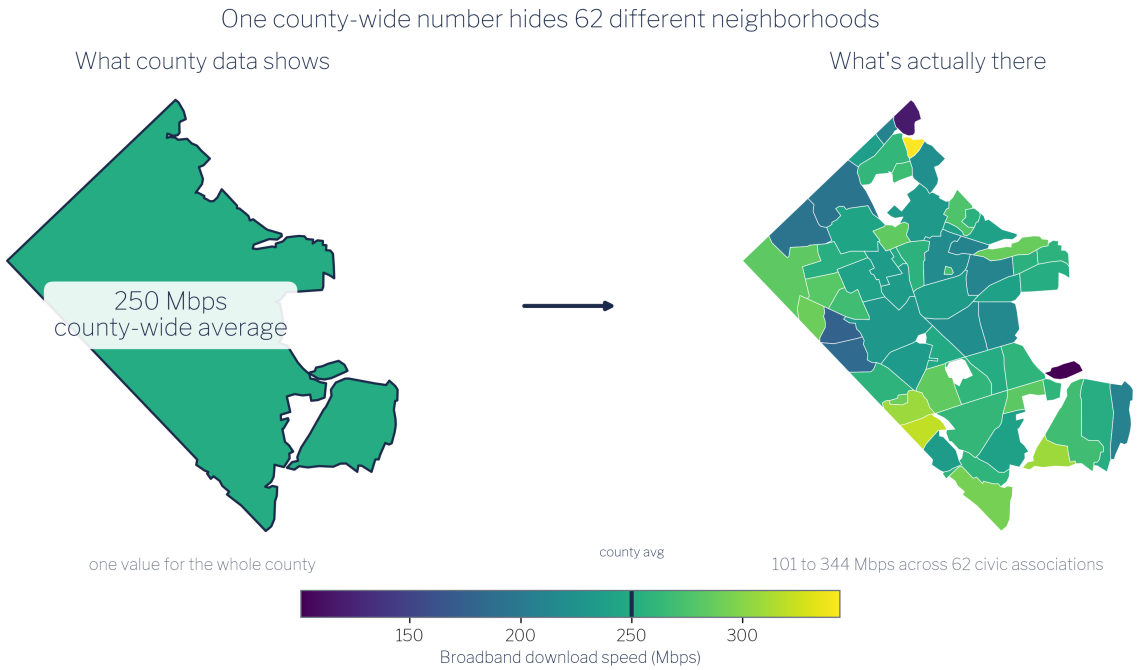


Figure 1.1: The same county at two resolutions. Reported as a single county-wide figure (left), Arlington's broadband download speed is one number, about 250 Mbps. Estimated at the civic-association level (right), the same measure ranges from 101 to 344 Mbps across 62 neighborhoods. This guide builds that sub-county view from source data that was never collected at the neighborhood scale.

Local officials make decisions at the neighborhood level, but the data they rely on is usually reported at the county level. A county-wide broadband adoption rate, a county-wide income figure, a county-wide health outcome: these statistics obscure the variation that actually matters for policy. A neighborhood with near-universal high-speed service and an adjacent neighborhood with almost none look identical in a county aggregate. The officials who need to target investment, design interventions, or justify budget requests are working blind.

The units that drive real decisions are not counties. They are neighborhood associations, service districts, planning corridors, school catchment zones, and utility territories. These are the geographies where residents organize, where services are delivered, and where disparities become visible. County-level data cannot answer the questions local officials are actually asking.

1.1 Why sub-county data is scarce

The scarcity of reliable sub-county data has three overlapping causes. First, small-area estimates carry large statistical uncertainty: when a sample is divided into dozens of small geographic units, the counts per unit shrink, and margins of error grow until estimates become unreliable [1]. Second, statistical agencies and data providers apply geographic suppression or aggregation to protect individual confidentiality; the smaller the area, the more likely a cell will be suppressed [2]. Third, there is no standardized collection infrastructure at the sub-county level. Unlike counties or census tracts, neighborhood and civic-association boundaries are locally defined, vary across jurisdictions, and change over time, so no national data program routinely publishes estimates for them [3].

The result is a persistent data gap between the geographic scale at which data is collected and reported and the geographic scale at which local government operates. Analysts working inside local government (and researchers trying to support them) must construct the sub-county datasets they need rather than retrieve them from a shelf [4].

1.2 What this guide demonstrates

This guide works through a concrete example: estimating mean household income and fixed-broadband download speeds for each of Arlington County, Virginia's 62 civic associations, which together cover approximately 109,528 households. The two source datasets (Ookla fixed-broadband speed-test tiles and American Community Survey block-group income and household counts) arrive on entirely different geographic footprints that match neither each other nor the civic-association boundaries. The guide shows, step by step, how to align those footprints through areal-weighted interpolation and produce estimates at the policy-relevant geography.

The worked example is implemented in both Python and R. The two produce the same estimates; each stage later in the guide shows the code in both languages, and the complete source for each is in the code appendices.

> **Who this guide is for**
>
> Local government analysts will find a reproducible workflow they can adapt to their own jurisdiction, data sources, and policy geographies. The method generalizes beyond broadband and income to any pair of spatially misaligned datasets.
> Decision-makers and program managers can read the prose and figures without engaging the code. The key takeaways are in the chapter narratives and maps. The technical sections are clearly marked.
> Some background in geographic data is helpful but not required. The guide explains each concept as it arises.

2 The Problem of Misaligned Boundaries

Every geographic dataset comes packaged in a particular set of spatial units: the shapes that define where each observation applies. Census estimates arrive at the block-group or tract level. Speed-test tiles come in a fixed web-mercator grid. Administrative records attach to parcels, precincts, or service zones. The problem is that the boundaries used to collect and publish one dataset almost never match those used for another, and neither typically matches the policy geography where decisions get made.

2.1 The Modifiable Areal Unit Problem

This mismatch has a formal name in spatial statistics: the Modifiable Areal Unit Problem, or MAUP [5]. The MAUP has two related components. The scale effect describes how statistical summaries (means, correlations, rates) change when the same underlying data is aggregated to larger or smaller units. The zonation effect describes how the same summary statistics change when you redraw boundaries at the same scale but in a different configuration. Neither effect is a modeling error; both are inherent properties of spatially aggregated data.

To see why, consider a single variable measured across one small area and summarised under two district schemes (Figure 2.1). The underlying values are identical in both cases; only the boundaries differ. One scheme cuts across the gradient and reports a nearly uniform figure of about 220 Mbps in every district, erasing the variation entirely. The other aligns with the gradient and reports districts ranging from 138 to 302 Mbps. Same data, opposite conclusions, with nothing changed but where the lines were drawn.

Figure 2.1: The Modifiable Areal Unit Problem in miniature. A single east-to-west speed gradient (left) is summarised under two district schemes covering the identical area. Boundaries A (centre) cut across the gradient, so every district averages about 220 Mbps and the variation disappears. Boundaries B (right) align with the gradient, so district averages range from 138 to 302 Mbps. The choice of geography is an analytical decision, not a clerical one.

The practical consequence for local policy work is significant: the relationship you observe between two variables can change substantially depending solely on the geographic units you use to measure it [6]. A correlation between income and broadband access that appears strong at the block-group level may weaken, strengthen, or reverse at the civic-association level, not because the underlying relationship changed, but because the boundary choice changed. Analysts who do not account for this can draw conclusions that are artifacts of the data's packaging rather than reflections of reality.

2.2 Three misaligned geographies

This guide works with three source geographies, none of which aligns with the others.

Ookla speed-test tiles are delivered on a zoom-level-16 web-mercator grid. Each tile is approximately 610 meters on a side: a fixed, global grid with no relationship to any political or

administrative boundary. Tiles do not respect county lines, block-group edges, or neighborhood borders.

ACS block groups are the Census Bureau's smallest standard geography for which the five-year American Community Survey publishes reliable estimates. Block groups within Arlington County range widely in area and population density; their boundaries follow streets, water features, and other physical markers that predate any local policy unit.

Arlington civic associations are community-defined neighborhoods and the county's formal channels for resident input and local governance. Their 62 boundaries were drawn through local political and historical processes, not by any statistical agency, and they bear no systematic relationship to the Census hierarchy.

The figure below illustrates the transformation this guide performs: taking source data packaged in two incompatible grids and producing estimates on the policy-relevant civic-association geography.

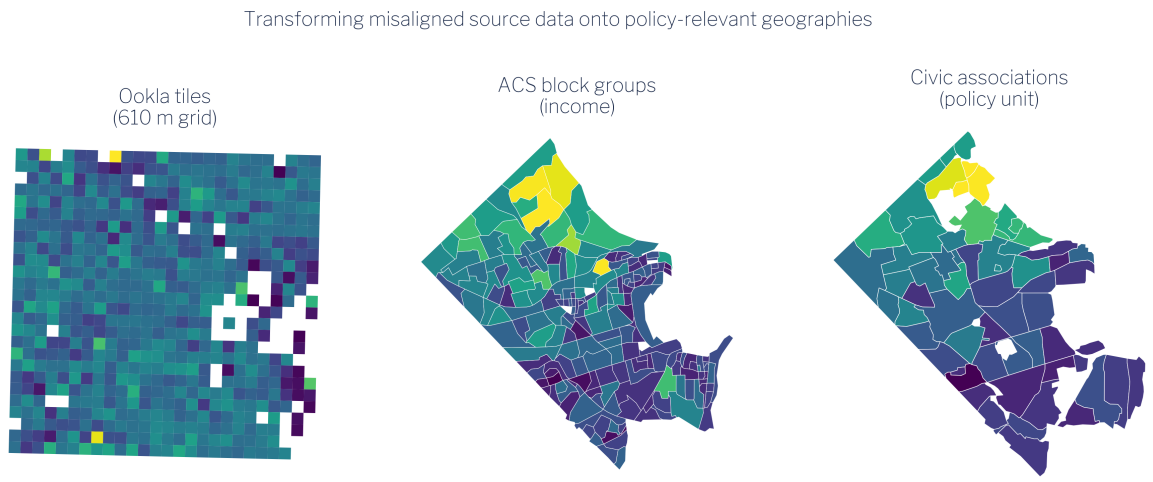


Figure 2.2: Transforming misaligned source data (Ookla tiles, ACS block groups) onto the policy-relevant civic-association geography.

As Figure 2.2 shows, a single civic association may overlap dozens of Ookla tiles and several block groups simultaneously. No one-to-one correspondence exists between any pair of these three geographies. The method described in subsequent chapters, areal-weighted interpolation, handles this overlap systematically by distributing each source observation across target units in proportion to the share of its area that falls within each target unit.

The key assumption underlying this approach is that the quantity of interest is distributed uniformly within each source unit. That assumption is never perfectly true, but it is a principled and auditable starting point, and far preferable to ignoring the boundary problem altogether.

3 The Data

This chapter describes the three datasets used in the worked example: Ookla fixed-broadband speed-test tiles, American Community Survey block-group income and household counts, and Arlington County civic-association boundaries. Each is publicly available.

3.1 Ookla Speed-Test Tiles

Ookla publishes quarterly snapshots of aggregated speed-test performance through its open-data program [7], [8]. The fixed-broadband dataset contains one record per zoom-level-16 web-mercator tile, a global grid in which each cell is approximately 610 meters on a side at mid-latitudes. For each tile that received at least one test in a quarter, the dataset reports the median download speed (Mbps), median upload speed (Mbps), and number of tests that contributed to those medians.

For this guide, the relevant quarter is Q1 2021, chosen to align with the ACS 2021 five-year estimates. Download speed and test count are the primary variables of interest. Test count matters because tiles with very few tests carry more uncertainty; the method chapter discusses how this is handled.

Ookla speed-test data reflects the experience of users who actively ran a test at Speedtest.net or through an embedded Speedtest SDK during the quarter. This is not a random sample of all internet users, and coverage varies with population density and smartphone penetration. These limitations are discussed further in the limitations chapter.

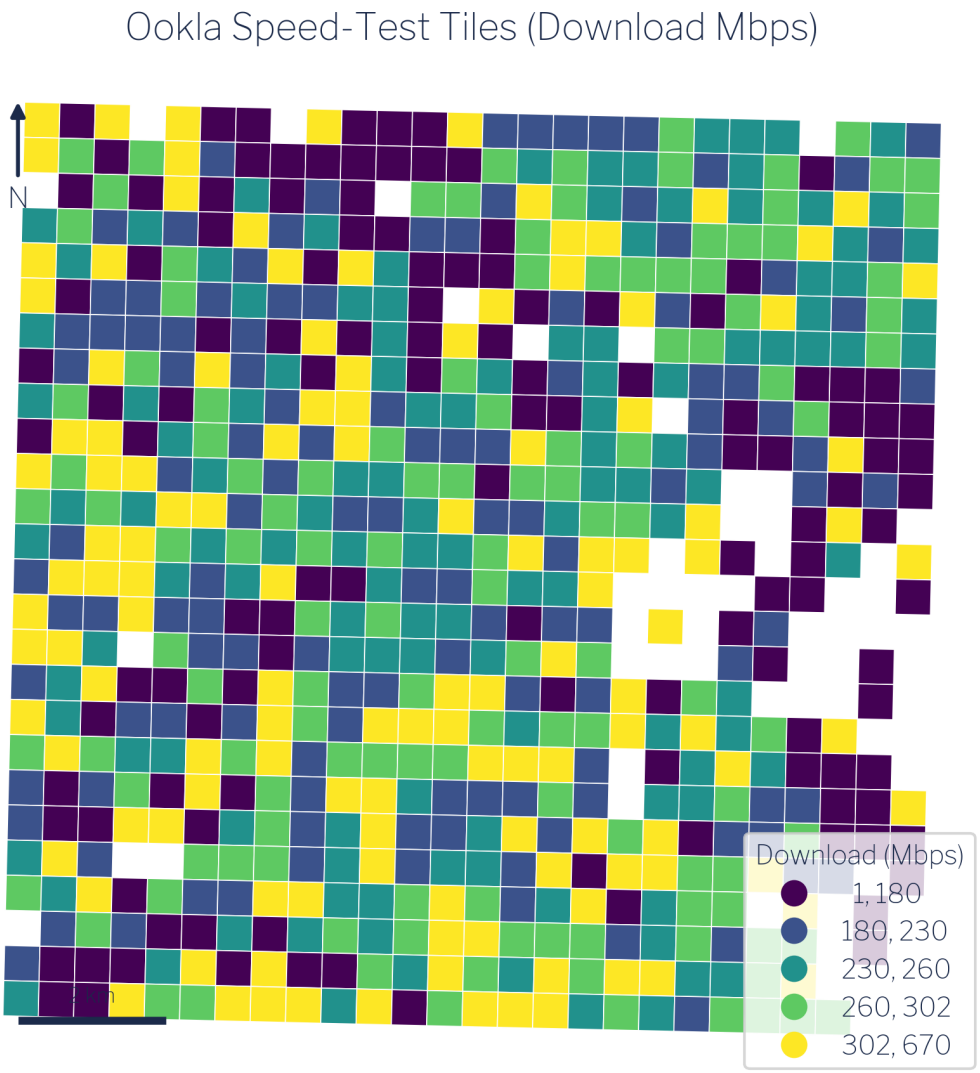


Figure 3.1: Ookla fixed-broadband speed-test tiles over Arlington (download Mbps).

3.2 American Community Survey

The American Community Survey is the Census Bureau's ongoing household survey, published annually as one-year and five-year estimates. The five-year estimates average data collected over a rolling 60-month window and are the appropriate choice for small geographies where single-year samples are too thin to support reliable estimates [9], [10].

This guide uses two block-group-level variables from the ACS 2021 five-year release:

- Table B19025: Aggregate household income. This is the sum of all household incomes within the block group, expressed in dollars. Unlike the median, aggregate income is an additive quantity: totals from sub-areas can be summed to produce a total for any containing area.
- Table B11001: Household count. The number of occupied housing units in the block group.

We use these two counts together rather than median income because mean income for any custom geography can be derived directly from them: divide aggregate income by household count. This arithmetic is only valid for additive counts, not medians. Using aggregate income and household counts preserves the ability to produce correct estimates at any aggregated geographic unit.

3.3 Arlington Civic Associations

Arlington County, Virginia is divided into 62 civic associations: community-defined neighborhoods that are the county's primary formal mechanism for organized resident input [11], [12]. Each association elects officers and engages with county government on land use, transportation, infrastructure, and community services. Membership in the Arlington County Civic Federation, which coordinates the associations, is the standard means by which neighborhoods participate in county decision-making.

The civic-association boundaries used here come from Arlington County's GIS open data portal [13]. The 62 associations cover the entire county and tile it without overlap, making them a complete and non-overlapping partition of Arlington's approximately 109,528 households. This makes them an ideal policy geography: they are locally meaningful, formally recognized, and together exhaustive.

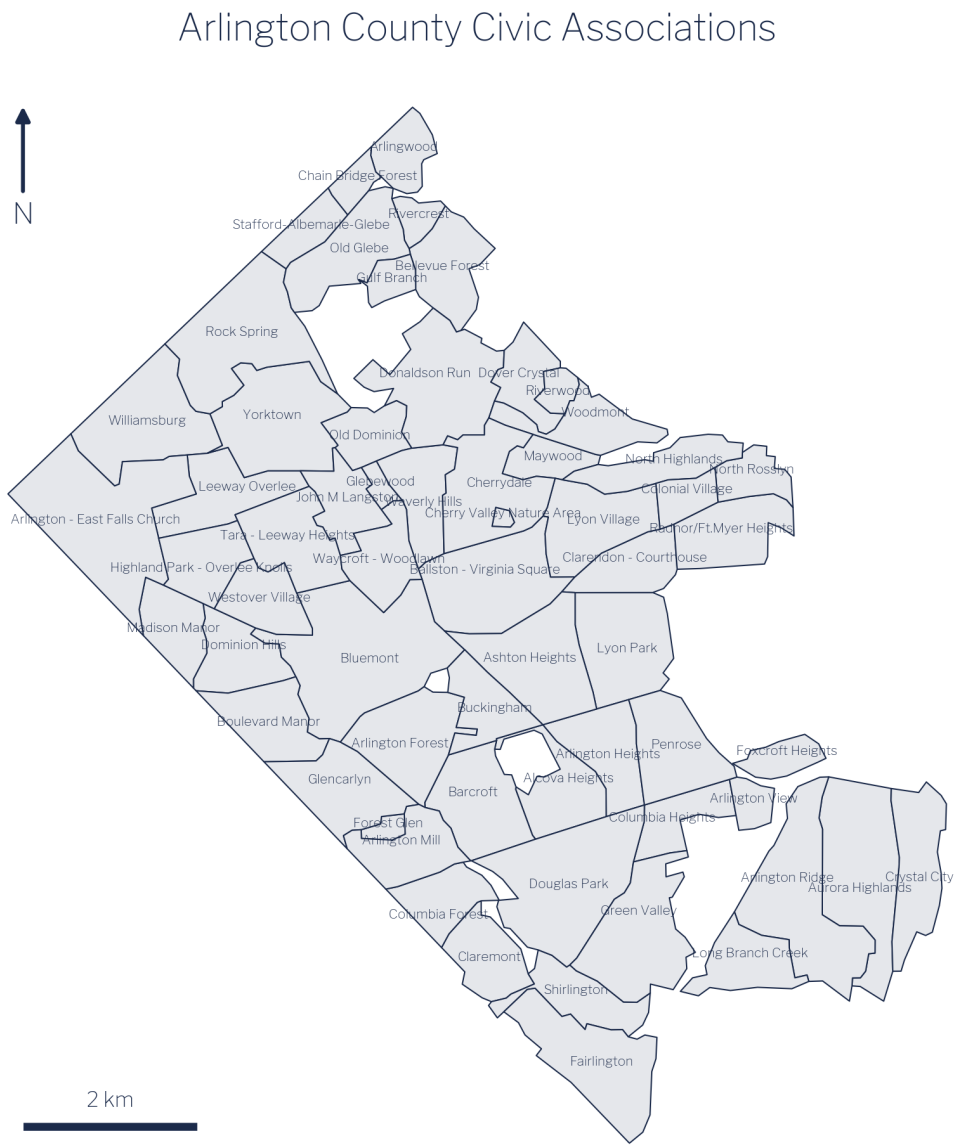


Figure 3.2: Arlington's 62 civic associations, the policy-relevant target geography.

Because civic associations are not part of the Census geographic hierarchy, no federal statistical agency publishes estimates at this level. Producing those estimates is precisely what the remainder of this guide demonstrates.

4 The Core Method: Areal Interpolation

The fundamental challenge of local-level data work is geometric: you have numbers attached to one set of polygons and questions about a different set of polygons. Areal interpolation is the family of methods that bridges that gap. The specific technique used throughout this guide, area-weighted redistribution, is the most transparent and widely applicable member of that family [14], [15].

4.1 How area-weighted redistribution works

The governing idea is simple: if a source area's polygon overlaps 40% of a target area's polygon, then 40% of that source area's count is assigned to that target area. More precisely, for each pair of source and target polygons that intersect, the method computes the fractional overlap (the ratio of intersection area to source area) and multiplies the source value by that fraction. When a target polygon overlaps multiple source polygons, the contributions from each source are summed to produce the target estimate.

Formally, the weight w_{ij} applied when transferring a value from source polygon i to target polygon j is:

$$w_{ij} = \frac{\text{area}(i \cap j)}{\text{area}(i)}$$

The weights across all target polygons that share area with source polygon i sum to 1.0, so the total value in the source layer is conserved. This is the key constraint that distinguishes redistribution from other forms of spatial estimation: values are allocated, not created.

The method rests on one explicit assumption: the variable of interest is distributed uniformly within each source polygon. Population is not perfectly uniform within a Census block group,

and download speed is not perfectly uniform across a 610-meter Ookla tile. But the assumption is transparent, auditable, and consistent, and it systematically outperforms simply ignoring the boundary problem altogether [16].

4.2 Extensive vs. intensive measures

This is the most important concept in the entire chapter. Understanding it determines whether your redistributed estimates are correct or subtly wrong.

Geographic measures divide into two fundamentally different types based on how they relate to the size of the area they describe.

Extensive measures (also called additive or count-like measures) scale with area. If you split a block group in two, each half has roughly half the total households, roughly half the aggregate income, roughly half the test count. Extensive measures can be legitimately redistributed using area weights because the splitting and summing is arithmetically valid. Examples: total households, aggregate household income, total speed-test count.

Intensive measures (also called rate-like or derived measures) do not scale with area. Mean income is the same whether you measure it for the whole block group or for a one-block sliver of it. If you split a block group and assign each half 40% of its mean income, you have made a category error: you are treating a rate as if it were a count. Examples: mean income, mean download speed, population density, any ratio.

The practical rule is therefore:

> Redistribute counts. Derive rates afterward.

To produce a valid mean income for a civic association, you redistribute aggregate income (extensive) and household count (extensive) separately to each civic association, then divide: mean income $=$ agg. income $\div$ households. You never redistribute mean income directly.

The same logic applies to a statistic that analysts sometimes want but cannot obtain this way: the median. Median household income is fundamentally a positional statistic (it describes the midpoint of a distribution) and it has no additive decomposition. There is no arithmetic

operation on two partial medians that recovers the median of their union. If you areally redistribute a median, the result is numerically meaningless. This guide uses aggregate income precisely because medians cannot be handled this way.

Getting this distinction wrong is the single most common error in areal interpolation. It produces estimates that look plausible but are quietly incorrect, typically biased toward area-weighted means of the rate rather than the population-weighted mean that the underlying counts would produce. In areas with uneven population density (Arlington has denser urban corridors alongside lower-density suburban neighborhoods), this error can be substantial.

Figure 4.1 summarizes the full redistribution workflow: extensive counts enter the redistribution, area weights are applied, and intensive rates are computed only at the end from the redistributed count totals.

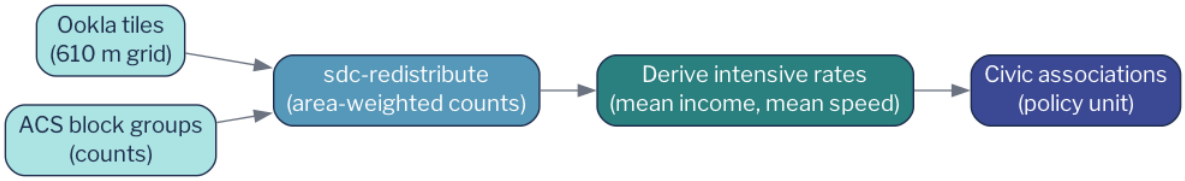


Figure 4.1: The redistribution method: extensive counts move area-weighted; intensive rates are derived from them.

4.3 The `sdc-redistribute` implementation

In practice, the redistribution is performed by the `sdc_redistribute` Python package, which handles the geometric intersection, weight calculation, and column-wise multiplication in a single call. The source geometry and one or more target geometries are supplied as GeoJSON or GeoDataFrame objects; the caller specifies which columns are extensive counts. The function returns a target-geometry GeoDataFrame with redistributed count columns that can be combined or divided to produce any desired derived measure.

Python: the redistribution call

```python
from sdc_redistribute import redistribute_direct

result = redistribute_direct(
    source_df=counts_long,            # geoid, year, measure, value
    source_geo="block_groups.geojson",
    target_geos={"civic_association": "civic_assoc.geojson"},
    count_cols=["agg_income", "households"],   # EXTENSIVE counts
    source_id="geoid",
)
# mean income (INTENSIVE) is derived afterwards:
#   mean_income = agg_income / households
```

The collapsible snippet above shows the essential pattern: only extensive counts are passed to `count_cols`. The intensive quantity, mean income, is not a parameter to the function; it is computed by the analyst on the output. This design makes the extensive-vs-intensive distinction architecturally explicit: the function cannot produce a rate; it can only redistribute counts.

The worked example in Chapter 5 applies this pattern twice in sequence, once for income and once for broadband speed, and then validates that the redistributed totals are internally consistent before proceeding to analysis.

5 Worked Example: Arlington, Step by Step

This chapter walks the full pipeline from raw source files to validated civic-association estimates. The five stages (acquire, redistribute income, redistribute broadband, combine, and validate) correspond directly to the pipeline modules in the companion repository. By the end, each of Arlington's 62 civic associations has an estimated mean household income and an estimated mean download speed derived from the same underlying geography and the same redistribution logic.

The complete runnable pipeline is a single command: `uv run python -m pipeline.run`.

5.1 Stage 1: Acquire the source data

Three datasets must be in hand before any redistribution can take place.

Census block-group boundaries are fetched programmatically using `pygris`, a Python wrapper around the Census Bureau's TIGER/Line shapefiles API. The call retrieves the block-group polygons for Arlington County (FIPS 51013) at the 2021 vintage, which aligns with the ACS five-year estimates used for income. The resulting GeoDataFrame provides the source geometry for the ACS redistribution step.

ACS income variables are retrieved through the Census API using a Census API key. The two variables used are B19025_001E (aggregate household income in the past 12 months) and B11001_001E (number of occupied housing units, i.e., households). Both are block-group-level estimates from the ACS 2021 five-year release. Neither is a median; both are extensive counts that can be legitimately redistributed. A Census API key is required; the pipeline reads it from the environment variable `CENSUS_API_KEY`.

Ookla fixed-broadband speed-test tiles are downloaded from Ookla's public Amazon S3 bucket, which hosts quarterly snapshots of the aggregated tile data as geoparquet files. The

pipeline fetches the Q1 2021 tile for the relevant S3 path, clips it to the Arlington bounding box, and retains two columns: `tests` (the count of speed tests in each tile during the quarter) and `avg_d_kbps` (the average download speed across those tests, in kilobits per second). Both are used in the broadband redistribution below.

With these three inputs in hand (block-group polygons, ACS counts, and Ookla tiles), the pipeline is ready to redistribute.

5.2 Stage 2: Redistribute income

Income follows the pattern described in Section 4.2. The two ACS variables, aggregate household income and household count, are extensive measures. Both are redistributed independently to civic associations using area-weighted interpolation. The redistribution preserves their totals: whatever aggregate income exists across all block groups flows, in its entirety, to the set of civic associations (since civic associations tile the county without gaps or overlaps).

Figure 5.1 illustrates the transformation. The source (block groups) carries two count columns, and the target (civic associations) receives redistributed versions of both.

Figure 5.1: Income transform: redistribute aggregate income + households, then derive mean income.

Once redistribution is complete, mean household income for each civic association is a simple arithmetic step:

$$\text{mean income}_j = \frac{\text{agg. income}_j}{\text{households}_j}$$

5.3 Python

```python
from sdc_redistribute import redistribute_direct
# agg_income and households are extensive counts.
res = redistribute_direct(counts_long, source_geo=block_groups,
                          target_geos={"civic_association": civic}, count_cols=["agg_income",
civic["mean_income"] = civic["agg_income"] / civic["households"]
```

5.4 R

```r
library(sf); library(sdc.redistribute)
# agg_income and households are extensive counts.
civic <- redistribute_direct(block_groups, civic,
                             extensive = c("agg_income", "households"))
civic$mean_income <- civic$agg_income / civic$households
```

The result is a population-weighted mean: larger block groups (more households) contribute more to the civic association's estimate than smaller ones, scaled continuously by the fraction of their area that falls within the association's boundary.

5.5 Stage 3: Redistribute broadband

Broadband speed requires one additional preparatory step because download speed, like income, is an intensive measure. The variable reported in the Ookla tiles, `avg_d_kbps`, cannot be directly redistributed. The count that makes it meaningful, `tests`, can.

The solution is to reconstruct the extensive quantity that, when divided by test count, recovers average speed. For each tile:

$$\text{speed} \times \text{tests} = \text{total download kilobits (proxy)}$$

This product is an extensive count: if a tile's area is split in two, each half contributed roughly half the total download kilobits. It can be redistributed. The test count is also extensive and can be redistributed. After both are moved to civic associations, the intensive rate is derived:

$$\text{download speed}_j = \frac{(\text{speed} \times \text{tests})_j}{\text{tests}_j} \div 1000$$

The division by 1,000 converts kilobits per second to megabits per second for presentation.

Figure 5.2 shows the broadband transform. Notice the structural parallel with the income transform in Figure 5.1: in both cases, the intensive output (mean income, mean speed) appears only after redistribution, never before.

Figure 5.2: Broadband transform: redistribute tests and speed×tests, then derive count-weighted speed.

5.6 Python

```python
# Speed is INTENSIVE: redistribute counts, then derive the rate.
tiles["d_product"] = tiles["avg_d_kbps"] * tiles["tests"]   # extensive
# ... redistribute tests + d_product to civic associations ...
civic["download_mbps"] = (civic["d_product"] / civic["tests"]) / 1000
```

5.7 R

```r
# Speed is INTENSIVE: redistribute counts, then derive the rate.
tiles$d_product <- tiles$avg_d_kbps * tiles$tests            # extensive
civic <- redistribute_direct(tiles, civic, extensive = c("tests", "d_product"))
civic$download_mbps <- (civic$d_product / civic$tests) / 1000
```

The snippet above captures the key lines. Creating `d_product` before redistribution is the move that makes everything else valid. An analyst who skipped this step and redistributed `avg_d_kbps` directly would produce a speed estimate weighted by area, not by test count: a different and less meaningful quantity that systematically overstates speed in low-density tiles with few tests.

5.8 Stage 4: Combine

With both redistributions complete, the income and broadband results are joined to the civic-association GeoDataFrame on the association identifier. The four intermediate columns (redistributed aggregate income, redistributed households, redistributed test count, and redistributed speed-product) are used to compute the two final output columns: `mean_income` and `download_mbps`. All 62 civic associations receive a value for both measures.

5.9 Stage 5: Validate

Before proceeding to analysis or visualization, the pipeline verifies one internal consistency check: household totals should be preserved by the redistribution. The sum of redistributed households across all 62 civic associations is compared to the sum of ACS household counts across all block groups. In a well-functioning redistribution over a complete, non-overlapping partition of the county (which Arlington's civic associations are), these totals should be identical up to floating-point rounding.

In practice, the redistributed household total is preserved within 2% of the source total, a result of minor edge effects at the county boundary where tile clipping introduces small geometric imprecision. This tolerance is well within the margin of the ACS sampling error itself, and it confirms that the redistribution has not created or destroyed households at any meaningful scale.

Analysts applying this method to their own geographies should perform this same check. A discrepancy larger than a few percent typically indicates either a non-exhaustive target partition (gaps between target polygons), a coordinate-reference-system mismatch between source and target layers, or a geometric validity problem in one of the input files.

With validation passed, the pipeline writes its output and the data is ready for mapping and analysis, the subject of the next chapter.

6 Results & Interpretation

The pipeline described in the preceding chapters produces two estimates for each of Arlington's 62 civic associations: a mean household income derived from ACS block-group counts, and a mean download speed derived from Ookla speed-test tiles. This chapter presents both distributions, compares them, and draws out the central finding, which runs counter to the most common assumption analysts bring to digital-divide work.

6.1 Income distribution

Mean household income across the 62 associations spans from roughly $73,000 (Arlington Mill) to $468,000 (Gulf Branch), with a mean of association means near $215,600 and a median near $191,500. The roughly 109,500 households represented in the data are not evenly spread. Arlington's geography concentrates its highest incomes in low-density, single-family neighbourhoods in the northern and western portions of the county: Gulf Branch, Rivercrest, Bellevue Forest, and Old Glebe all exceed $447,000. Denser, more mixed-use corridors in the south and east (Columbia Heights, Arlington Mill, Forest Glen) cluster toward the lower end.

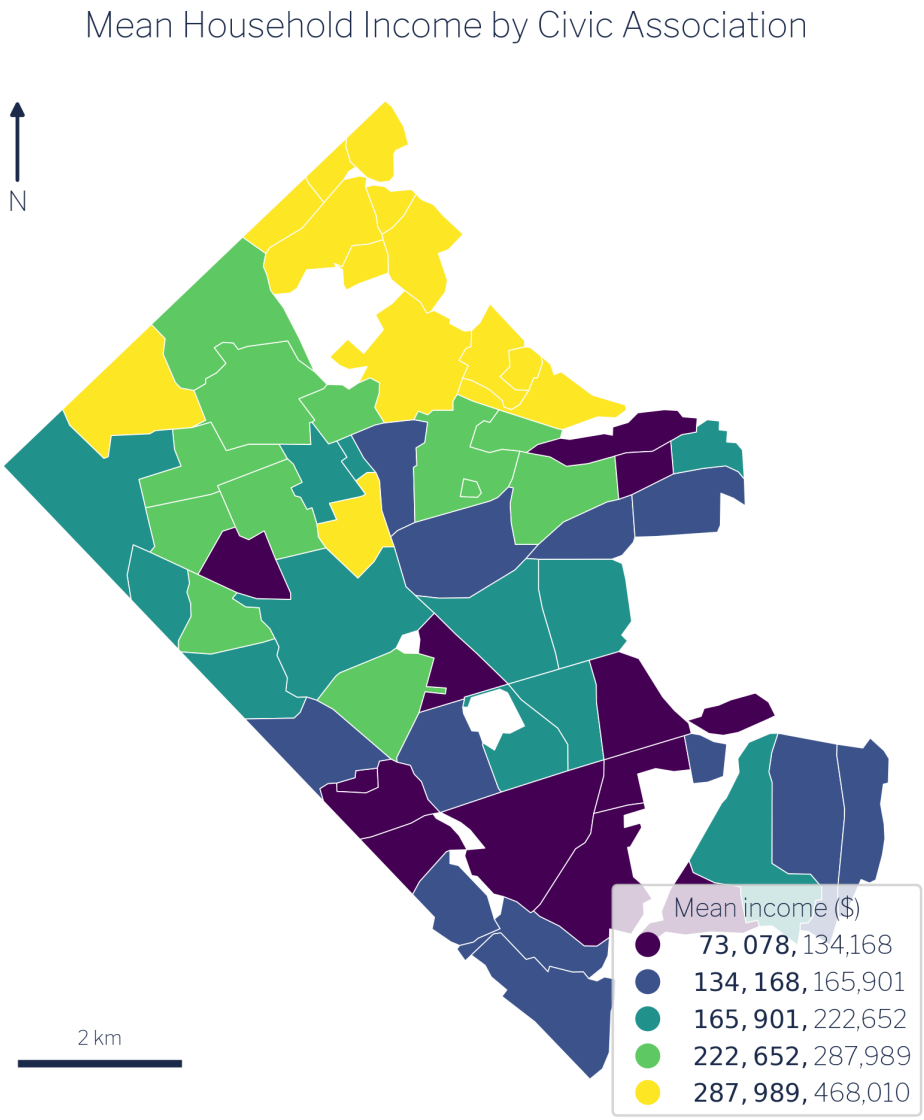


Figure 6.1: Mean household income by civic association.

The spatial pattern in Figure 6.1 reflects Arlington's well-documented socio-spatial structure: income rises sharply as one moves away from the Rosslyn–Ballston and Route 1 corridors into the low-density residential tracts that border Fairfax and Falls Church [17]. The spread is wide for a jurisdiction that ranks among the wealthiest counties in the United States, which makes the county an instructive case: if income structured broadband access anywhere, it should be detectable here.

6.2 Broadband speed distribution

Mean download speed across the same 62 associations ranges from 100.8 Mbps (Foxcroft Heights) to 320.8 Mbps (Columbia Forest), with a county-wide mean of approximately 246.8 Mbps and a median of 251.9 Mbps. Mean upload speed averages roughly 105 Mbps. Even at the low end, every association exceeds the FCC's legacy 25/3 Mbps benchmark; the real differentiation occurs at the higher tiers that support heavy simultaneous use.

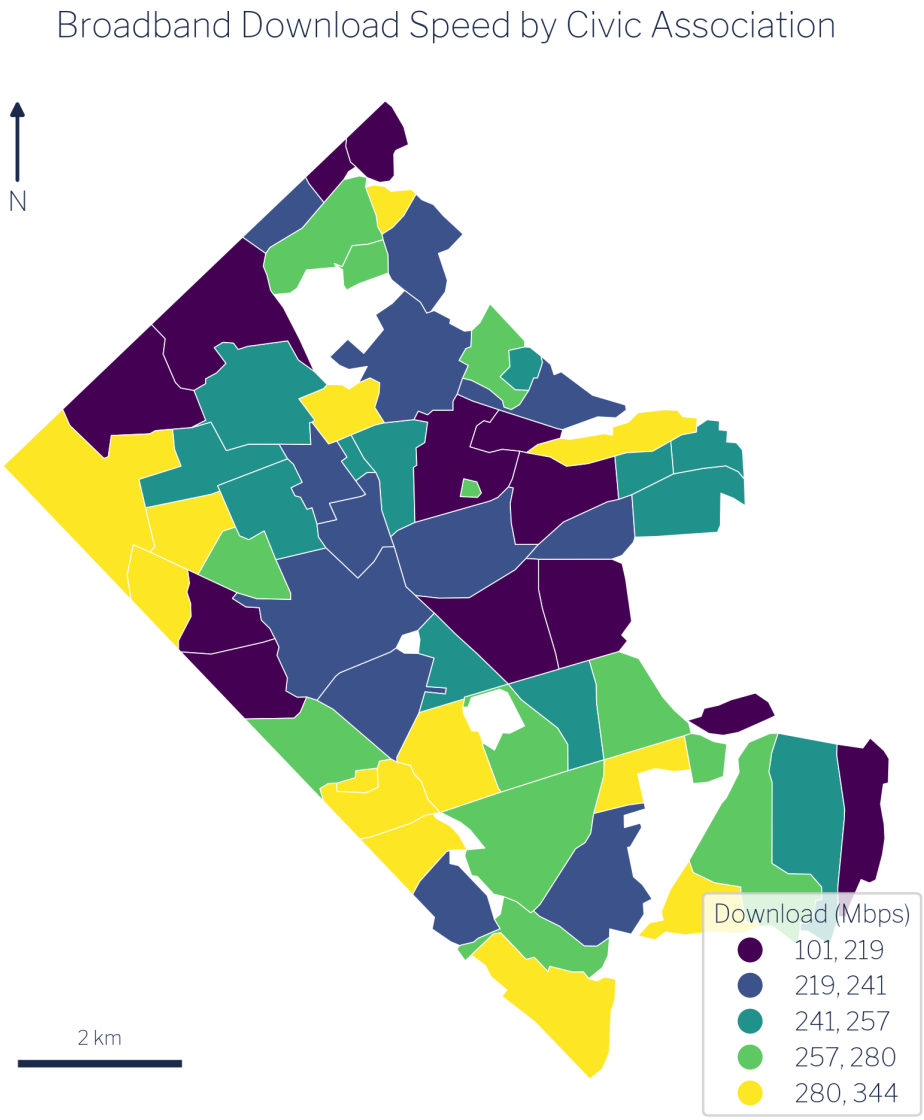


Figure 6.2: Broadband download speed by civic association.

The spatial pattern in Figure 6.2 is striking and largely inverts the income map. The fastest associations are concentrated in the denser southern and central corridors. Columbia Forest reaches 320.8 Mbps; Long Branch Creek 307.9 Mbps; Arlington Mill, the lowest-income association in the dataset at approximately $73,000, records the third-fastest speed at 307.0 Mbps. Fairlington follows at 293.4 Mbps. Conversely, the slowest associations (Foxcroft Heights at 100.8 Mbps, Arlingwood at 117.3 Mbps, Dominion Hills at 176.2 Mbps) are not low-

income enclaves; they are mid-to-high-income, low-density, single-family neighbourhoods in the county's outer northern tier.

6.3 The central finding: income and speed are decoupled

The Pearson correlation between mean household income and mean download speed across the 62 civic associations is r = −0.11. This value is substantively indistinguishable from zero. Income and broadband speed are essentially decoupled in Arlington.

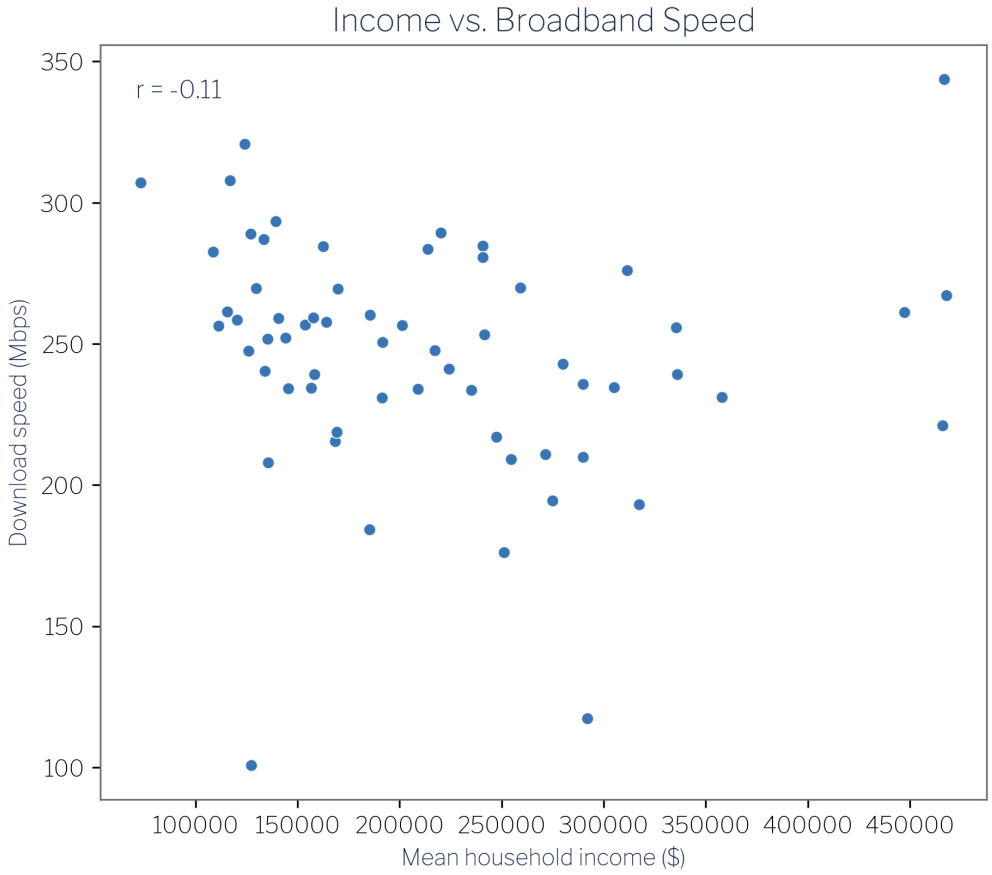


Figure 6.3: Income vs. broadband download speed across the 62 civic associations (r = −0.11).

The scatter plot in Figure 6.3 makes the absence of a relationship visually plain. There is no line, positive or negative, that summarises the cloud in any useful way. Analysts who arrive

at this data expecting the standard "rich neighbourhoods get faster internet" finding will not find it here. The naive expectation, plausible on its face and documented in national-scale studies of access gaps [18], does not hold inside a uniformly affluent jurisdiction where income variation is high but where the key driver of infrastructure investment operates on a different dimension entirely.

That dimension is housing density. Dense multifamily corridors generate the subscriber concentration that makes competitive fibre and cable deployment commercially attractive to internet service providers. Arlington's Rosslyn–Ballston spine and the Route 1 corridor have both density and the infrastructure investment that follows from it. Low-density single-family enclaves in northern Arlington have neither the building stock to attract the same level of competitive investment nor the resulting speeds, regardless of how wealthy their residents are.

6.4 Income-to-speed ratio

To quantify the mismatch between income and infrastructure, consider the income-to-speed ratio: dollars of mean household income per Mbps of mean download speed. Across the 62 associations this ratio ranges from $238 per Mbps (Arlington Mill, high speed, lower income) to $2,490 per Mbps (Arlingwood, low speed, higher income), more than a ten-fold span.

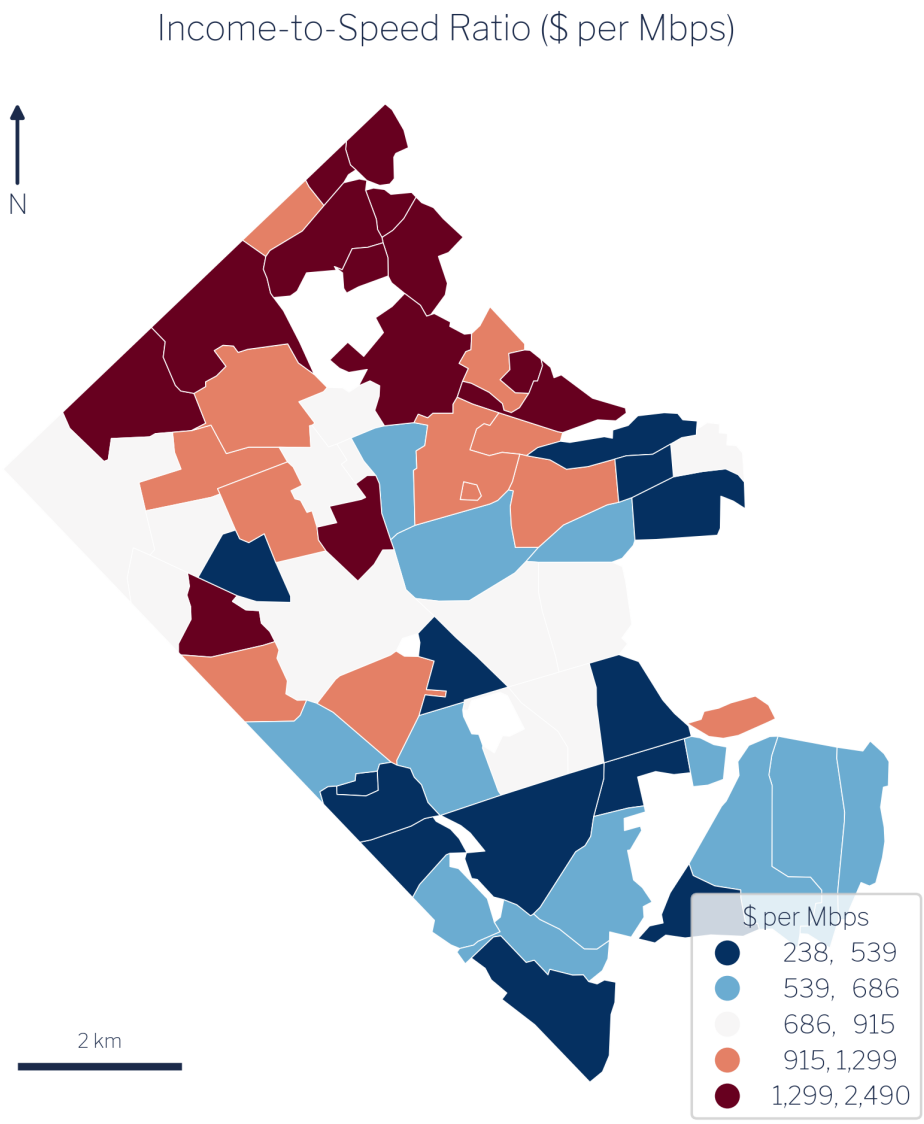


Figure 6.4: Income-to-speed ratio (dollars of household income per Mbps of download speed).

The map in Figure 6.4 is effectively a map of which associations are getting the least broadband for their economic standing. The highest-ratio associations (Arlingwood at $2,490, Bellevue Forest at $2,107, Gulf Branch at $1,752, Old Glebe at $1,713) are all wealthy, low-density, northern-tier communities. They are not suffering a crisis of unaffordability; they are simply not served by the same density of competitive infrastructure as their

southern counterparts. The lowest-ratio association, Arlington Mill, has the strongest infrastructure relative to its income level precisely because dense multifamily housing attracted competitive carriers.

6.5 Bivariate synthesis

Classifying each association by income tercile (low/mid/high) and speed tercile (low/mid/high) produces a three-by-three contingency table that summarises the joint distribution. The cell counts are:

	Low speed	Mid speed	High speed
Low income	4	8	9
Mid income	6	7	7
High income	11	5	5

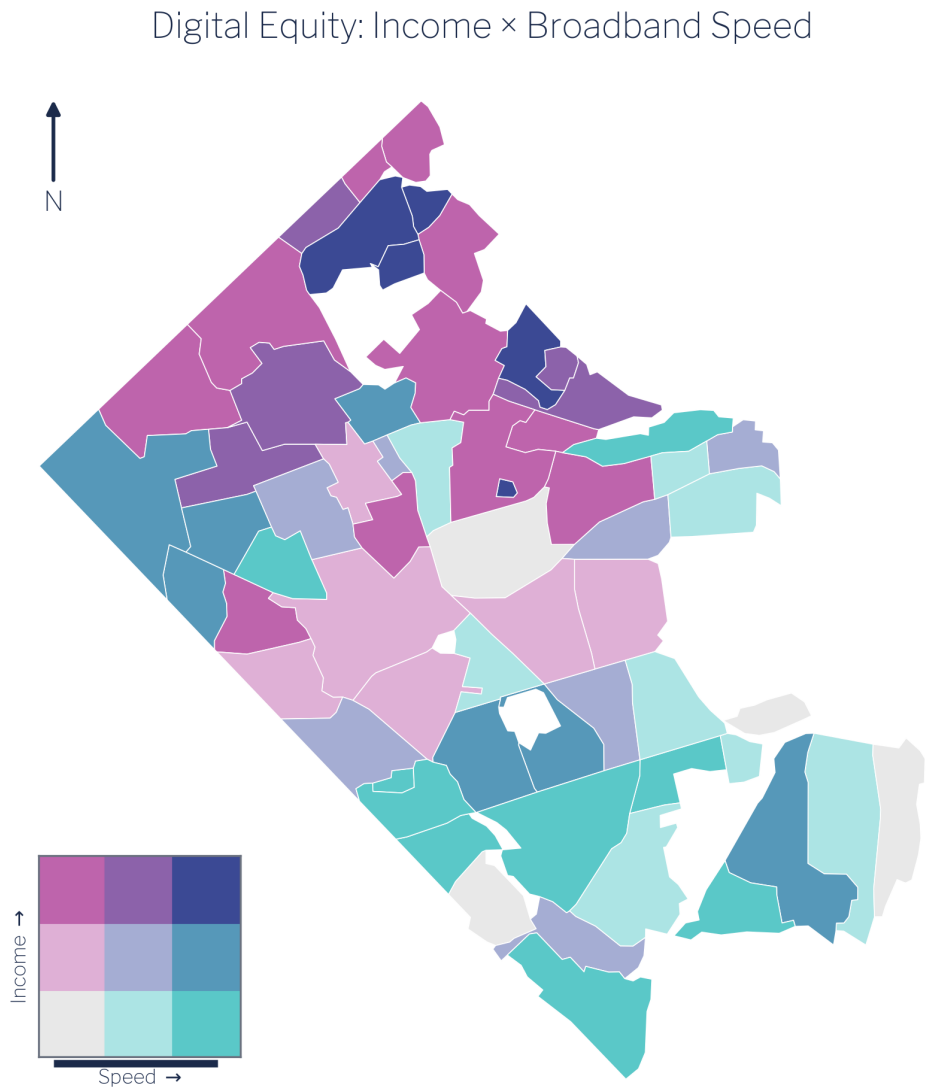


Figure 6.5: Bivariate map: income tercile × speed tercile across the 62 civic associations.

The largest single cell is high-income / low-speed, with 11 associations: Williamsburg, Maywood, Dominion Hills, Arlingwood, Bellevue Forest, Donaldson Run, and their neighbours. This cell alone falsifies the "rich areas get fast internet" hypothesis. At the opposite corner, 9 associations fall in the low-income / high-speed cell: Columbia Heights, Forest Glen, Columbia Forest, Westover Village, Long Branch Creek, and Arlington Mill, the dense multifamily corridors already identified as speed leaders.

The bivariate map in Figure 6.5 makes the geography legible. The high-income/low-speed cluster occupies the low-density northern and western portions of the county. The low-income/high-speed cluster wraps around the denser central and southern corridors. The pattern is structurally coherent and consistent across all three ways of visualising it: the scatter plot, the ratio map, and the bivariate class map.

6.6 Policy implications

The finding that the digital-divide axis in Arlington is housing density and infrastructure build-out, not household income, has direct implications for how jurisdictions should design broadband equity interventions [19], [20].

Income-based subsidy targeting, the most common policy tool, would direct resources toward low-income households while leaving the actual speed gaps unaddressed. The households that experience the weakest broadband performance in Arlington are, on average, high-income households in low-density neighbourhoods. An income-tested programme would systematically miss them.

More effective interventions would target the mechanism driving the gap: the commercial calculus that makes dense corridors attractive to competitive carriers but leaves single-family enclaves with fewer options. This suggests policy levers such as right-of-way facilitation for fibre providers in low-density zones, municipal broadband franchising conditioned on area coverage, and infrastructure requirements built into residential subdivision permitting. None of these is calibrated to household income; all are calibrated to the density-and-competition gap that the data actually reveal.

The Arlington case cannot be assumed to generalise directly to jurisdictions with lower overall incomes or with genuine gaps in physical access infrastructure. But the methodological lesson applies everywhere: verify the relationship between income and speed empirically before designing income-based interventions.

7 A Second Approach: Parcel-Based Redistribution

The area-weighted method introduced in Chapter 4 works well, but it rests on a simplifying assumption worth examining: that each variable of interest is distributed uniformly across a block group's entire polygon. In practice, a block group's polygon includes roads, parks, parking lots, commercial strips, and other land that contains no housing at all. For measures tied to where people actually live, this matters. Dasymetric methods replace the uniform-distribution assumption by using an ancillary layer (in this case, parcel locations) to concentrate redistributed values where housing actually sits [16].

This chapter implements that alternative and then asks directly: how much does the choice of method change the result?

7.1 The parcel data

Arlington County's Master Housing Unit Database (MHUD) provides a parcel-level inventory of every residential unit in the county. The layer used here contains 34,340 parcels carrying roughly 127,000 housing units in total. The vast majority of those parcels (33,665) are single-family detached or attached (SFD/SFA) structures, typically carrying one or two units apiece. The remaining 675 parcels are classified as multifamily (MULTI), and they account for a disproportionate share of the county's housing stock: the multifamily parcels are concentrated along the Rosslyn–Ballston corridor and the Route 1 spine, where high-rise and mid-rise buildings each contribute hundreds of units to the total.

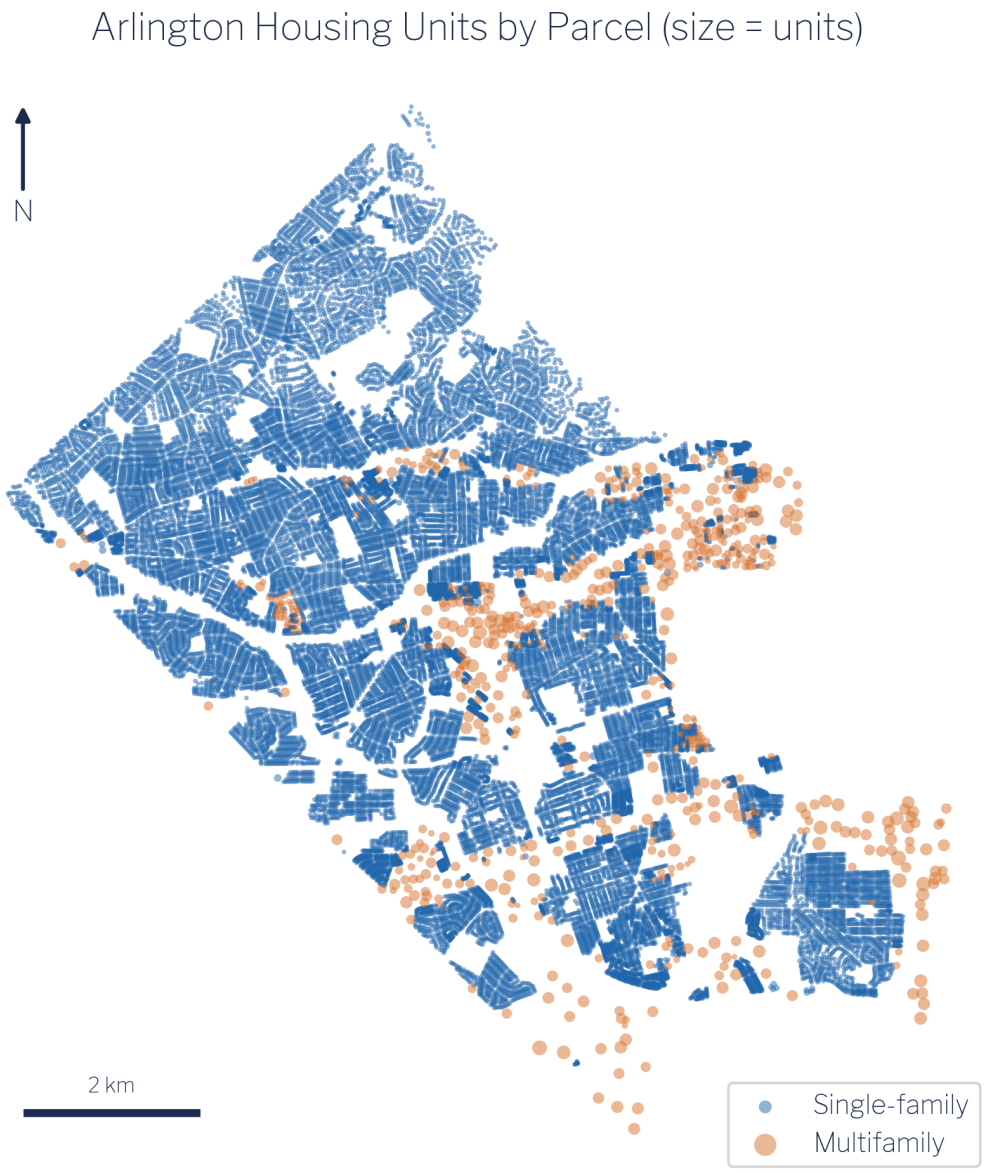


Figure 7.1: Arlington housing units by parcel. Point size is unit count; the few multifamily parcels carry most of the county's housing units, concentrated in the Rosslyn–Ballston and Route 1 corridors.

Figure 7.1 makes the spatial concentration vivid. A small number of large dots, each representing a multifamily building, account for a large fraction of the county's residential footprint. The single-family parcels are numerous but individually small, spread across

the county's lower-density northern and western neighbourhoods. This heterogeneity is precisely what dasymetric redistribution is designed to exploit: instead of spreading a block group's household counts uniformly across its entire polygon, we concentrate them where the housing units are.

7.2 The method: unit-weighted redistribution

The implementation follows a straightforward unit-weighting scheme. Each parcel is "exploded" into a number of identical point records equal to its `Total_Units` count: a 300-unit multifamily building contributes 300 identical centroid points, while a single-family parcel contributes one. That pool of unit points is then passed to `redistribute_parcels` from the `sdc-redistribute` package, which uses the point locations as the dasymetric control layer. The block-group aggregate income and household counts are redistributed to civic associations in proportion to the number of unit points that fall within each association, weighted by their source block-group membership.

Figure 7.2 shows the spatial logic. A source block group's total is divided across the parcels inside it, and those parcels are then collected under whatever target boundary we care about, here a civic association that overlaps the block group only partially. Only the parcels falling inside the target contribute to its estimate.

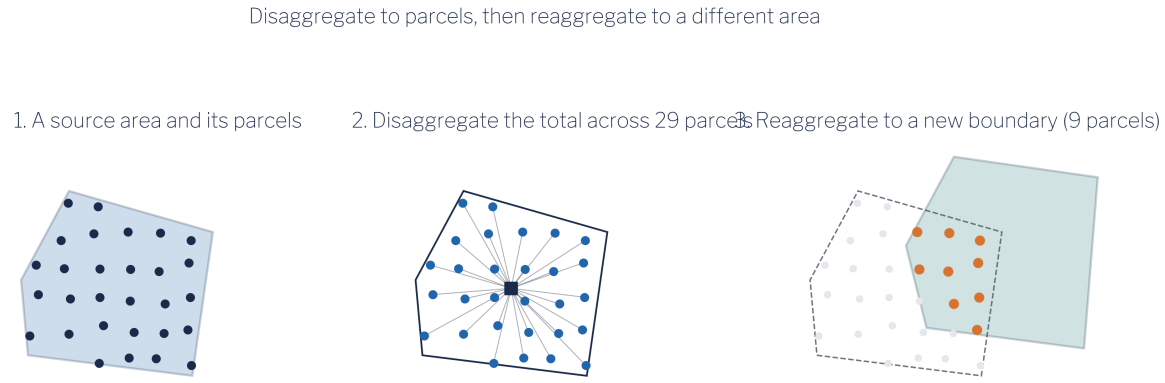


Figure 7.2: Parcel-based redistribution in three steps. A source area and its parcels (1); the source total disaggregated across those parcels (2); and the parcels reaggregated under a different target boundary, where only the parcels inside the target (highlighted) contribute to its estimate (3).

7.3 Python

```python
from sdc_redistribute import redistribute_parcels
# explode each parcel into Total_Units identical points first, then:
result = redistribute_parcels(
    source_df=counts_long, parcel_centroids=unit_points,
    source_geo="block_groups.geojson",
    target_geos={"civic_association": "civic_assoc.geojson"},
    count_cols=["agg_income", "households"], source_id="geoid",
)
```

7.4 R

```r
library(sf); library(sdc.redistribute)
# Split each block group's counts across its parcels, weighted by unit count.
civic <- redistribute_parcels(block_groups, civic, parcels,
                              extensive = c("agg_income", "households"),
                              weights = "Total_Units")
civic$mean_income <- civic$agg_income / civic$households
```

The logic is the same as the area-weighted approach (redistribute extensive counts, derive intensive rates afterward), with one structural difference: the implicit population surface has changed from a uniform density within each block group to a density proportional to the spatial distribution of housing units within that block group.

7.5 Results: a close agreement with the area-weighted estimates

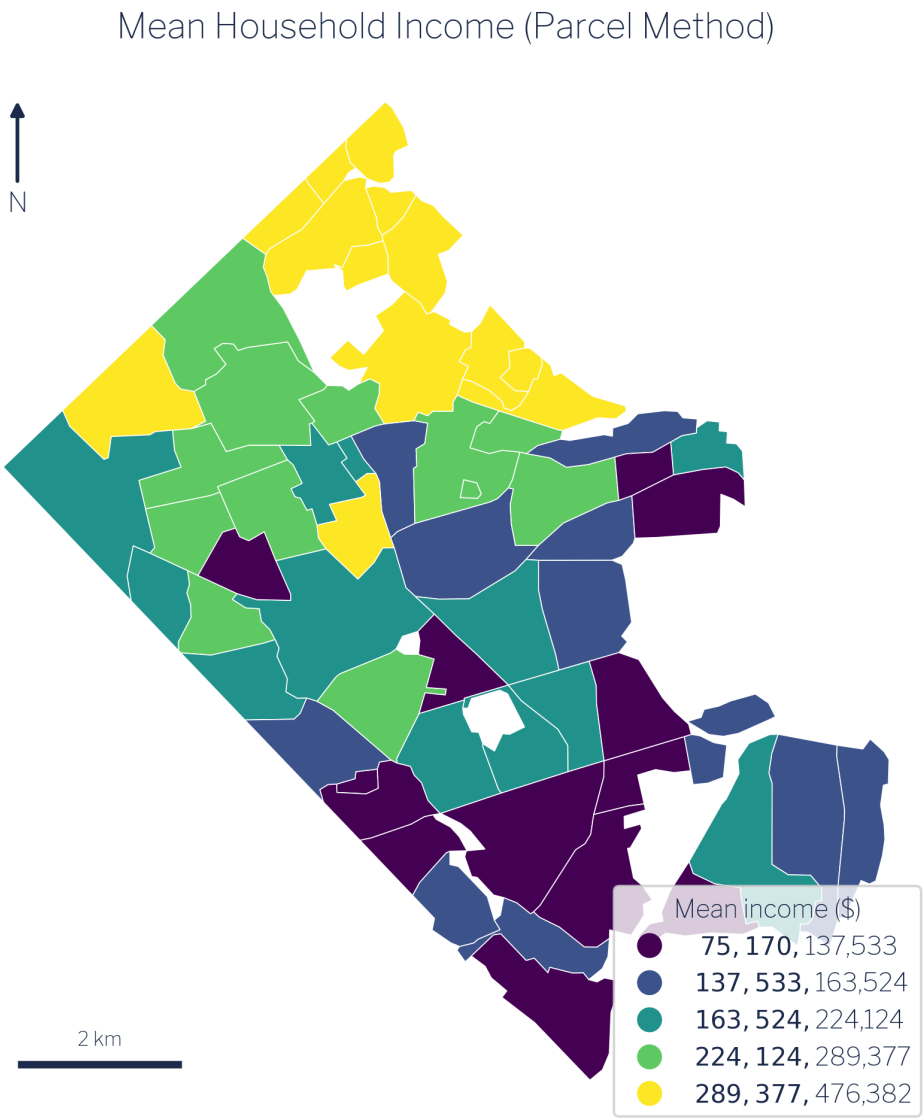


Figure 7.3: Mean household income by civic association, parcel-based redistribution.

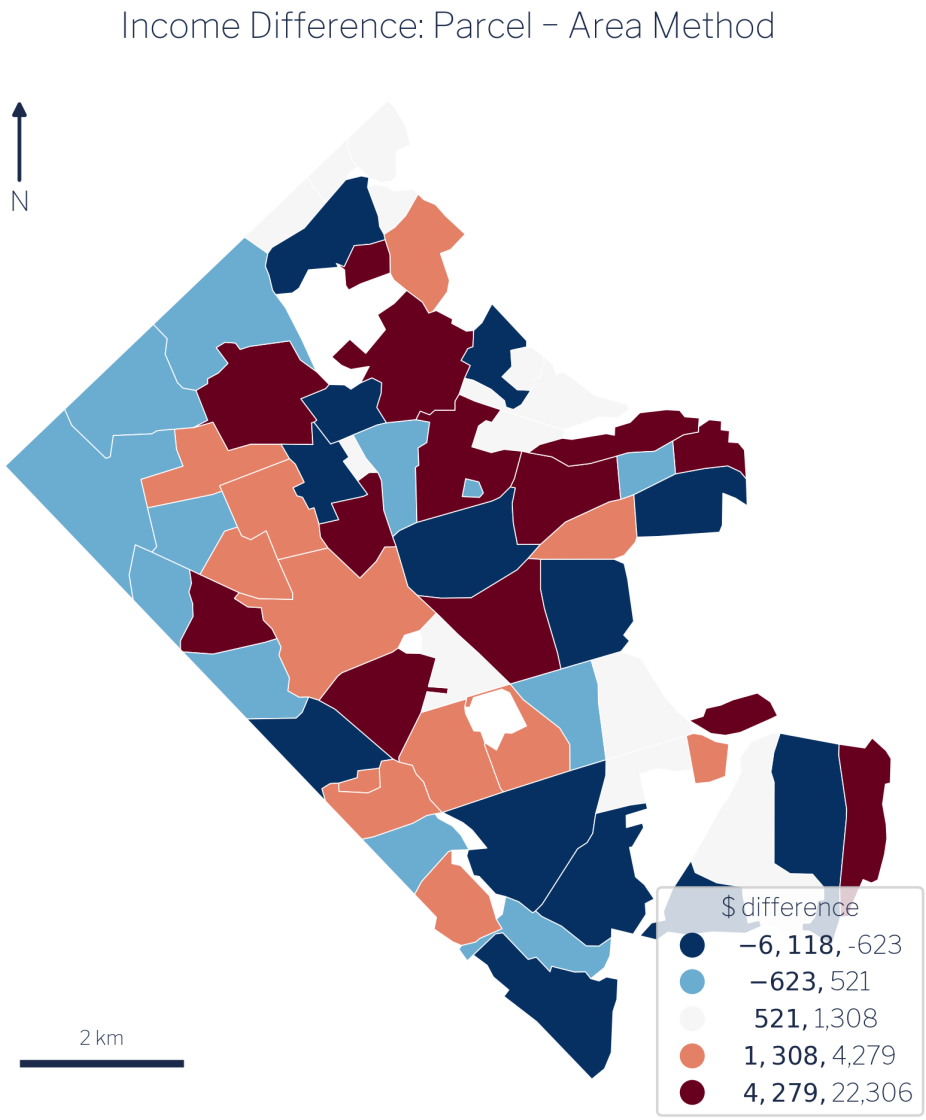


Figure 7.4: Difference in mean household income: parcel minus area-weighted (dollars).

Figure 7.3 and Figure 7.4 tell a consistent story. The parcel map looks nearly identical to its area-weighted counterpart, and the difference map is dominated by values close to zero.

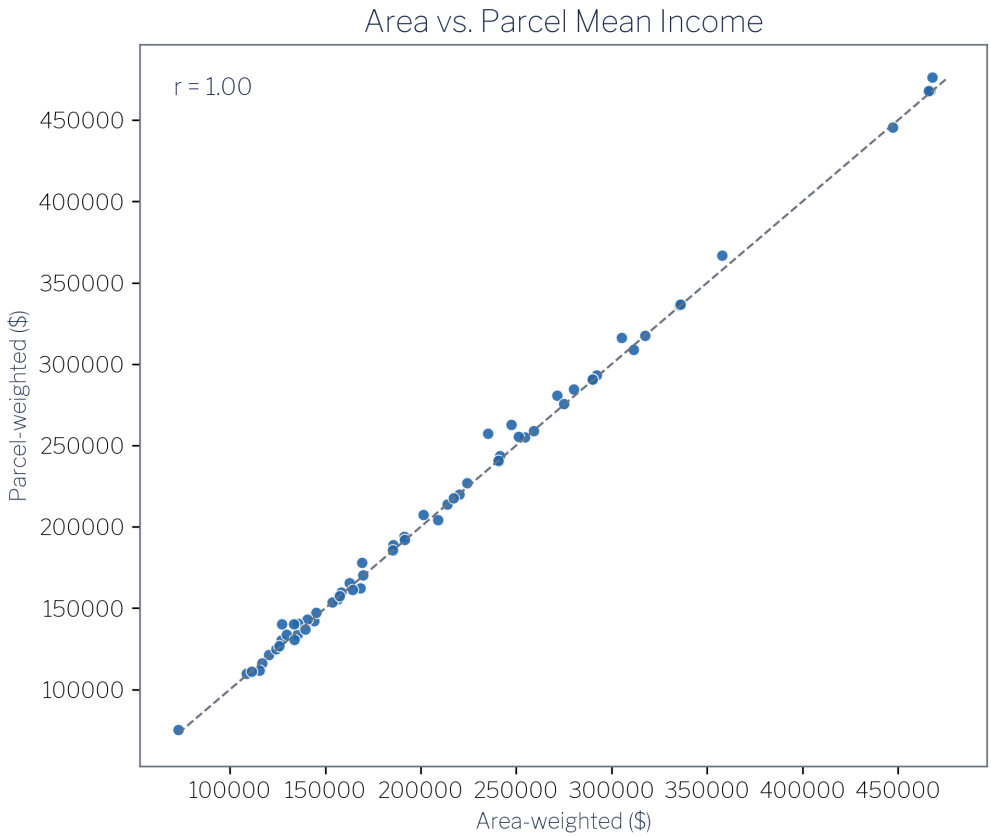


Figure 7.5: Area-weighted vs. parcel-weighted mean income across the 62 civic associations.

The scatter plot in Figure 7.5 makes the relationship explicit: the two methods produce a Pearson r = 1.00 across the 62 associations. The largest per-association shift in either direction is approximately ±$22,000, a few percent on six-figure incomes. The three biggest gainers under parcel weighting are Arlington Forest (+$22,306), Cherrydale (+$15,347), and Foxcroft Heights (+$13,047). The three biggest losers are Lyon Park (−$6,118), John M Langston (−$4,960), and Douglas Park (−$3,781).

Why do the methods agree so closely? Mean household income is a smooth, intensive measure: it describes a characteristic of households, not a count of things that accumulate over area. When a block group's income is fairly evenly distributed among its housing stock, as it generally is in Arlington at the block-group scale, then where within the block group the housing sits barely changes the weighted average that reaches each civic association. Parcel placement shifts modest fractions of the income count from one association to an adjacent

one, but because the income levels on both sides of any given boundary are similar, the effect on the mean nearly cancels. The ±$22,000 maximum shift represents the cases where a multifamily building with an unusually different household composition sits near a boundary: a real effect, but a small one relative to the income range in the data.

7.6 Leakage and the case for parcel weighting on count measures

One honest limitation deserves direct statement. Approximately 3,032 households (about 2.8% of the county total) sit in parcels whose centroids fall outside any civic association boundary. These households contribute to the block-group totals but are not allocated to any association under the parcel method. The parcel approach therefore slightly underestimates total households compared with the area-weighted method, which allocates every square metre of a block group to some association. Analysts using this method for official estimates should be aware of this leakage and decide whether to redistribute orphaned parcels to the nearest association or treat them as a separate "unassigned" category.

For mean income, which is what this guide measures, the 2.8% household leakage has a negligible effect on the ratio. For count measures (total population, total housing units), the answer is different. Dasymetric and parcel-based redistribution earn their keep precisely when the placement of housing within a block group drives the allocation of counts across target boundaries. If you want to know how many housing units fall within a specific civic association's polygon, point-in-polygon counting on the parcel layer is more accurate than area weighting; the difference can be substantial in associations with compact boundaries or with multifamily buildings near their edges.

The natural next step in this direction is household-size weighting: single-family and multifamily units have systematically different average household sizes, and a redistribution that treats all units as interchangeable conflates these. The `sdc-redistribute` package's `redistribute_parcel_pums_adj` function implements this extension by drawing on Public Use Microdata Sample (PUMS) household-size distributions, stratified by unit type, to weight single-family parcels differently from multifamily ones. Applying that method to Arlington's MHUD data, and assessing whether the household-size correction changes the civic-association estimates meaningfully, is a planned extension of this work.

For the income analysis in this guide, the two methods are interchangeable in practice. The parcel approach is presented here to establish the infrastructure and to make a broader point:

method choice matters most when the variable in question is a count and when housing is unevenly distributed within source units. When the variable is a smooth intensive measure like mean income, and when the redistribution is operating at the block-group-to-neighbourhood scale, area weighting and parcel weighting converge.

8 Limitations & Uncertainty

The results in Chapter 6 are derived estimates, not direct measurements. Every step of the pipeline, from the source data to the redistribution to the final ratio calculations, introduces uncertainty. This chapter itemises the four most important sources and explains what each implies for interpreting and extending the analysis.

8.1 Area-weighting assumes uniform internal distribution

The core assumption of area-weighted interpolation is that the quantity being redistributed is distributed uniformly within each source unit. For ACS income, the source units are census block groups; for Ookla broadband, the source units are 610-metre speed-test tiles. Wherever that uniformity assumption is wrong, wherever households or speed-test activity are spatially clustered within a unit, the redistribution will misallocate a fraction of the value.

In practice, both income and population density vary within block groups, particularly at the edges where block groups straddle the civic-association boundary. The effect is largest for target associations that are small relative to the source units they overlap, and smallest for large associations that wholly contain many source units. Analysts should treat estimates for the smallest associations with additional caution and, where possible, validate against parcel-level or point-level data.

This limitation is not unique to this workflow; it is inherent to any area-weighted interpolation approach [14], [21]. The `redistribute_direct` function used here, like all area-weighting methods, cannot recover sub-unit spatial variation that is not encoded in the source data.

8.2 Ookla data reflects active test-takers, not all households

The Ookla speed-test tiles record the average performance of users who actively ran a speed test during the measurement quarter. This introduces a form of self-selection bias: test-takers are not a random sample of all internet subscribers. People who run speed tests are more likely to be technically engaged, more likely to be troubleshooting a slow connection, and possibly more concentrated in particular building types or demographic groups [22].

The direction of this bias is not straightforwardly predictable. If frustrated users test more frequently, the Ookla data may understate typical speeds. If technically sophisticated users dominate the test pool, the data may overstate typical speeds. For the purposes of a comparative analysis across civic associations within a single county, what matters is whether the bias is spatially systematic: whether the test-taker population differs in structure across associations in ways that would distort the between-association comparisons. That cannot be fully evaluated without ground-truth data.

Users of this analysis should treat the download speeds as indicative of infrastructure capacity at the high end of the user experience distribution, not as estimates of median or typical household experience.

8.3 ACS income carries sampling error

The ACS five-year block-group estimates are not administrative records; they are survey estimates with associated margins of error. At the block-group level, particularly for smaller block groups, the coefficient of variation on income variables can be substantial. The guide uses aggregate household income (B19025_001E) and household count (B11001_001E) rather than median income precisely because both are extensive, summable measures that survive redistribution with interpretable units.

However, the mean income ultimately derived for each civic association is still the ratio of two redistributed survey estimates. Both the numerator (aggregate income) and denominator (household count) carry ACS sampling uncertainty. In well-populated block groups this uncertainty is modest; in smaller block groups it can be large enough to materially affect the redistributed estimate. Arlington's relatively high response rates and the use of five-year pooled estimates (2017–2021) mitigate but do not eliminate this concern [23].

The redistribution does not propagate formal uncertainty bounds. An analyst who needs confidence intervals on the civic-association estimates would need to request the block-group-level margins of error from the Census API and carry them through the redistribution algebra, a non-trivial extension that is beyond the scope of the current pipeline.

8.4 `redistribute_direct` rescales measures independently

The `redistribute_direct` function rescales each source column independently to ensure its total is preserved in the target geography. When two columns are redistributed independently and then combined into a ratio, as happens with both income (aggregate income / households) and broadband (speed-product / tests), the rescaling applied to each column need not be identical. Near the county boundary, where tile clipping and block-group edge effects are most pronounced, the two columns may be rescaled by slightly different factors, producing a ratio that is not perfectly equivalent to what a joint redistribution would yield.

In the Arlington validation, the household total is preserved within approximately 2% of the source total, well within the ACS sampling error and acceptable for a comparative county-level analysis. For applications where ratio precision at the county boundary is critical, analysts should consider masking associations that have more than a threshold fraction of their area outside the source coverage, or running a sensitivity check that compares estimates from independent and joint redistribution paths.

9 Apply It to Your Own Jurisdiction

The Arlington analysis is a worked illustration of a general workflow. Any jurisdiction with civic associations, planning districts, school attendance zones, or other custom geographies can follow the same steps to estimate income and broadband speed (or any other combination of an ACS count variable and an Ookla speed-test variable) at the sub-county level. This chapter distils the pipeline into a numbered recipe and points to the tools needed to run it.

9.1 The six-step recipe

1. Define your target geography as a GeoJSON with a stable identifier. Your target polygons, the units you want estimates for, must be stored as a GeoJSON (or any format readable by GeoPandas) and must include a column that uniquely identifies each polygon. The companion repository uses `geoid` by convention, but any string or integer key works as long as it is stable and unique. The polygons must be in a standard coordinate reference system (WGS 84 / EPSG:4326 is the safe default) and should tile the jurisdiction without meaningful gaps or overlaps. Gaps will cause households and tests near the edges to be unallocated; overlaps will cause double-counting.

2. Obtain source counts on their native geography. For income, retrieve ACS block-group estimates for the county of interest via the Census API. The two variables needed are B19025_001E (aggregate household income) and B11001_001E (number of households). Both must be five-year estimates aligned with the same ACS vintage. For broadband, download the relevant Ookla quarterly fixed-broadband tile from Ookla's public S3 bucket. Clip the tile to your jurisdiction's bounding box to reduce memory usage. Retain `tests` and `avg_d_kbps`. Both datasets must be in the same CRS as your target geography before redistribution.

3. Redistribute the source counts to your target geography. Call `redistribute_direct` (from the `sda_areal` Python package, or its R equivalent once available) with your source Geo-

DataFrame and target GeoDataFrame. For ACS, redistribute `agg_income` and `households`. For Ookla, first construct `d_product = avg_d_kbps * tests`, then redistribute `tests` and `d_product`. Each call preserves the source total within floating-point tolerance.

4. Derive the intensive rates from the redistributed counts. Mean income is `agg_income_target / households_target`. Download speed in Mbps is `(d_product_target / tests_target) / 1000`. Neither derived variable should be redistributed directly; the correct workflow always redistributes the underlying counts and derives the rate afterward. Attempting to redistribute mean income or average speed directly will produce area-weighted results that are correct in only the degenerate case where all source units are identical in density.

5. Validate that totals are preserved. Before proceeding to analysis, compare the sum of redistributed households across all target polygons to the sum of ACS households across all source block groups. They should agree within a few percent. A larger discrepancy indicates a geometry problem (gaps, overlaps, CRS mismatch) that will bias every downstream estimate. Similarly check that redistributed test counts sum to within a few percent of source tile test counts. Both checks are implemented in the companion repository's `pipeline.validate` module.

6. Map, analyse, and interpret. Join the income and speed estimates to your target GeoDataFrame and export as GeoJSON, Parquet, or whatever format your visualisation or analysis tool requires. The companion repository builds all five result figures (choropleth maps, scatter plot, and bivariate map) through a single command.

9.2 Running the companion pipeline

The full pipeline is executed with a single command from the repository root:

```
uv run python -m pipeline.run
```

This command fetches the Census and Ookla data, performs both redistributions, validates totals, and writes the output GeoJSON. To regenerate the result figures from the pipeline output:

```
uv run python -m pipeline.build_figures
```

Both commands require `uv` (the Python package manager), a Census API key in the environment variable `CENSUS_API_KEY`, and an internet connection. The repository README provides setup instructions. All code is open-source and MIT-licensed.

9.3 R implementation

The areal interpolation logic in the Python pipeline mirrors the `sdc-redistribute` family of functions available in R. A runnable R implementation of the redistribution stages ships with this guide in `pipeline-r/`, built on the `sdc.redistribute` R package; the data-acquisition recipe uses `tigris` for Census geometries, `tidycensus` for ACS data, and `ooklaOpenDataR` for speed-test tiles. Analysts already working in R can follow the same six-step recipe; the redistribution algebra is language-agnostic.

9.4 Adapting to other variables

The recipe generalises beyond income and broadband. Any ACS extensive variable (counts of people, households, housing units, workers) can be redistributed in step 3 by substituting the appropriate B-table variables. Any Ookla variable that can be expressed as a count-weighted product (upload speed follows the same `avg_u_kbps * tests` construction as download speed) can be substituted in the same step. Analysts working with other speed-test or coverage datasets will need to identify the analogous extensive quantity (typically a count that, when summed and divided into a product column, recovers the intensive rate of interest) and follow the same pattern.

10 The Complete Pipeline Code (Python)

This appendix reproduces the full, runnable Python pipeline behind the worked example: the actual source, not a simplified sketch. It is generated directly from the `pipeline/` package in the companion repository, so it always matches the code that produced every figure and number in this guide.

The pipeline is built on the `sdc-redistribute` package and runs end to end with a single command:

```
uv run python -m pipeline.run
```

The modules below appear in execution order. Figure generation (`pipeline/style.py`, `pipeline/maps.py`, `pipeline/figures.py`) lives in the same repository and is omitted here to keep the focus on the data method.

10.1 Configuration

Central settings: the Arlington FIPS codes, the ACS count variables (aggregate income and households), and the dataset paths every other module reads.

`pipeline/config.py`

```
"""Central configuration: geographies, CRS, source identifiers, paths."""
from pathlib import Path

from dotenv import load_dotenv

# Load secrets (e.g. CENSUS_API_KEY) from a local .env if present. The file is
```

```
# gitignored; loading it here makes the key available to any pipeline module.
load_dotenv()

# Arlington County, Virginia
STATE_FIPS = "51"
COUNTY_FIPS = "013"
ACS_YEAR = 2021          # 5-year ACS vintage used throughout
OOKLA_YEAR = 2023
OOKLA_QUARTER = 2

# Area weighting is handled inside sdc-redistribute, which reprojects geographic
# inputs to EPSG:3857 before computing intersection areas. At county scale the
# choice of projected CRS is immaterial (<0.3% area-ratio spread vs State Plane),
# so the pipeline does not impose its own projected CRS.

# ACS variables are EXTENSIVE COUNTS (never redistribute the median directly).
# Mean household income is derived later as agg_income / households.
ACS_VARS = {
    "agg_income": "B19025_001",   # Aggregate household income (dollars)
    "households": "B11001_001",   # Total households (count)
}

# Ookla open data S3 parquet path template
OOKLA_S3 = (
    "s3://ookla-open-data/parquet/performance/type=fixed/"
    "year={year}/quarter={quarter}/"
    "{year}-{month:02d}-01_performance_fixed_tiles.parquet"
)

# Paths
ROOT = Path(__file__).resolve().parent.parent
DATA = ROOT / "data"

# Source / output dataset paths
CIVIC_ASSOC = DATA / "va013_geo_arl_2021_civic_associations.geojson"
```

```
BLOCK_GROUPS = DATA / "va013_block_groups.geojson"          # produced by acquire_geographies
BLOCKS = DATA / "va013_geo_blocks.geojson"                  # existing
ACS_COUNTS = DATA / "va013_acs_income_counts.geojson"       # produced by acquire_acs
OOKLA_TILES = DATA / "ookla_tiles_arlington.geojson"        # existing/refreshable
CIVIC_INCOME = DATA / "civic_income.csv"                    # produced by redistribute_income
CIVIC_BROADBAND = DATA / "civic_broadband.csv"             # produced by redistribute_broadban
CIVIC_COMBINED = DATA / "civic_combined.geojson"           # produced by combine
PARCELS = DATA / "va013_parcels.geojson"                   # produced by acquire_parcels
CIVIC_INCOME_PARCELS = DATA / "civic_income_parcels.csv"   # produced by redistribute_income_p
CIVIC_INCOME_COMPARISON = DATA / "civic_income_comparison.geojson"  # produced by compare_meth
```

10.2 Acquire census geographies

Download Arlington's census block groups, the source geography for the income redistribution.

`pipeline/acquire_geographies.py`

```
"""Acquire Arlington census block-group geometries via pygris."""
from __future__ import annotations

import geopandas as gpd
from pygris import block_groups

from pipeline import config


def fetch_block_groups() → gpd.GeoDataFrame:
    """Download Arlington County (VA/013) block groups for the ACS vintage."""
    bg = block_groups(
        state=config.STATE_FIPS,
        county=config.COUNTY_FIPS,
        year=config.ACS_YEAR,
        cb=True,
```

```
    )
    return bg


def main() -> None:
    bg = fetch_block_groups()
    bg.to_file(config.BLOCK_GROUPS, driver="GeoJSON")
    print(f"Wrote {len(bg)} block groups → {config.BLOCK_GROUPS}")


if __name__ == "__main__":
    main()
```

10.3 Acquire ACS income counts

Pull aggregate household income and household counts from the Census API. These are counts (extensive measures); mean income is derived later.

`pipeline/acquire_acs.py`

```
"""Acquire Arlington ACS aggregate income + household counts (block group)."""
from __future__ import annotations

import os

import geopandas as gpd
import pandas as pd
from census import Census

from pipeline import config
from pipeline.acquire_geographies import fetch_block_groups


def _sanitize_counts(df: pd.DataFrame) -> pd.DataFrame:
```

```
    """Replace Census negative sentinels (e.g. -666666666) with 0.

    Aggregate income and household counts are non-negative by definition; the
    Census API returns large negative sentinel codes for suppressed or
    not-applicable cells (typically zero-household block groups). Left in place,
    these sentinels would propagate through area-weighted redistribution and
    produce negative incomes. Treating them as 0 is correct for the
    zero-household areas they mark.
    """
    out = df.copy()
    for col in ("agg_income", "households"):
        out[col] = out[col].mask(out[col] < 0, 0)
    return out


def fetch_acs_counts(api_key: str | None = None) -> pd.DataFrame:
    """Return a DataFrame: GEOID, agg_income, households for Arlington block groups."""
    key = api_key or os.environ.get("CENSUS_API_KEY")
    c = Census(key, year=config.ACS_YEAR)
    # The census library needs the API estimate suffix "E" on each variable
    # (e.g. B19025_001E), unlike tidycensus which strips it.
    agg_var = config.ACS_VARS["agg_income"] + "E"
    hh_var = config.ACS_VARS["households"] + "E"
    rows = c.acs5.state_county_blockgroup(
        fields=("NAME", agg_var, hh_var),
        state_fips=config.STATE_FIPS,
        county_fips=config.COUNTY_FIPS,
        blockgroup=Census.ALL,
        tract=Census.ALL,
    )
    df = pd.DataFrame(rows)
    df["GEOID"] = df["state"] + df["county"] + df["tract"] + df["block group"]
    df = df.rename(columns={agg_var: "agg_income", hh_var: "households"})
    df = _sanitize_counts(df)
    return df[["GEOID", "agg_income", "households"]]
```

```
def main() -> None:
    counts = fetch_acs_counts()
    bg = fetch_block_groups()[["GEOID", "geometry"]]
    merged = bg.merge(counts, on="GEOID", how="left")
    gdf = gpd.GeoDataFrame(merged, geometry="geometry", crs=bg.crs)
    gdf.to_file(config.ACS_COUNTS, driver="GeoJSON")
    print(f"Wrote {len(gdf)} block groups with income counts → {config.ACS_COUNTS}")


if __name__ == "__main__":
    main()
```

10.4 Acquire Ookla broadband tiles

Read the Ookla open-data broadband tiles from S3 and clip them to Arlington.

`pipeline/acquire_ookla.py`

```
"""Acquire Ookla fixed-broadband tiles for Arlington from S3 open data."""
from __future__ import annotations

import geopandas as gpd
import pandas as pd
from shapely import wkt

from pipeline import config
from pipeline.acquire_geographies import fetch_block_groups


def _s3_path() -> str:
    month = config.OOKLA_QUARTER * 3 - 2  # Q1→01, Q2→04, Q3→07, Q4→10
    return config.OOKLA_S3.format(
        year=config.OOKLA_YEAR, quarter=config.OOKLA_QUARTER, month=month
```

```
    )


def fetch_ookla_tiles() -> gpd.GeoDataFrame:
    """Download and bbox-filter Ookla tiles to Arlington County."""
    df = pd.read_parquet(_s3_path(), storage_options={"anon": True})
    df["geometry"] = df["tile"].apply(wkt.loads)
    tiles = gpd.GeoDataFrame(df, geometry="geometry", crs="EPSG:4326")

    bbox = fetch_block_groups().to_crs("EPSG:4326").total_bounds  # minx,miny,maxx,maxy
    minx, miny, maxx, maxy = bbox
    return tiles.cx[minx:maxx, miny:maxy].copy()


def main() -> None:
    tiles = fetch_ookla_tiles()
    cols = ["quadkey", "avg_d_kbps", "avg_u_kbps", "avg_lat_ms", "tests", "devices", "geometry
    tiles[cols].to_file(config.OOKLA_TILES, driver="GeoJSON")
    print(f"Wrote {len(tiles)} Ookla tiles → {config.OOKLA_TILES}")


if __name__ == "__main__":
    main()
```

10.5 Redistribute income

Area-weight the income and household counts onto civic associations with `sdc-redistribute`, then derive mean household income.

`pipeline/redistribute_income.py`

```
"""Redistribute ACS income counts to civic associations and derive mean income."""
from __future__ import annotations
```

```
from pathlib import Path

import geopandas as gpd
import pandas as pd
from sdc_redistribute import redistribute_direct

from pipeline import config


def redistribute_income(
    counts_long: pd.DataFrame, source_geo: Path, target_geo: Path
) -> pd.DataFrame:
    """Return per-civic-association agg_income, households, and derived mean_income.

    ``counts_long`` is long format (geoid, year, measure, value) with measures
    ``agg_income`` and ``households``.
    """
    result = redistribute_direct(
        source_df=counts_long,
        source_geo=source_geo,
        target_geos={"civic_association": target_geo},
        count_cols=["agg_income", "households"],
        source_id="geoid",
    )
    # result is long with measures agg_income_direct / households_direct
    wide = result.pivot_table(
        index="geoid", columns="measure", values="value", aggfunc="first"
    ).reset_index()
    wide.columns.name = None
    wide = wide.rename(
        columns={"agg_income_direct": "agg_income", "households_direct": "households"}
    )
    # Mean household income is an INTENSIVE quantity derived from two counts.
    wide["mean_income"] = wide["agg_income"] / wide["households"].where(
        wide["households"] > 0
```

```
    )
    return wide[["geoid", "agg_income", "households", "mean_income"]]


def main() -> None:
    # ACS counts GeoJSON uses uppercase GEOID; rename to the lowercase `geoid`
    # that redistribute_direct expects, for both the long-format counts and the
    # source geometry. Write a geoid-keyed source file for redistribute_direct.
    bg = gpd.read_file(config.ACS_COUNTS).rename(columns={"GEOID": "geoid"})
    long = bg[["geoid", "agg_income", "households"]].melt(
        id_vars="geoid", var_name="measure", value_name="value"
    )
    long["year"] = config.ACS_YEAR

    src_path = config.DATA / "_acs_src.geojson"
    bg[["geoid", "geometry"]].to_file(src_path, driver="GeoJSON")
    out = redistribute_income(long, src_path, config.CIVIC_ASSOC)
    src_path.unlink(missing_ok=True)

    out.to_csv(config.CIVIC_INCOME, index=False)
    print(f"Wrote {len(out)} civic-association income rows → {config.CIVIC_INCOME}")


if __name__ == "__main__":
    main()
```

10.6 Redistribute broadband

Redistribute test counts and the speed×tests product, then derive the count-weighted mean download and upload speeds.

`pipeline/redistribute_broadband.py`

```
"""Redistribute Ookla broadband to civic associations; derive mean speeds."""
from __future__ import annotations

from pathlib import Path

import geopandas as gpd
import pandas as pd
from sdc_redistribute import redistribute_direct

from pipeline import config

MIN_TESTS = 5  # statistical-reliability filter on source tiles


def redistribute_broadband(
    counts_long: pd.DataFrame, source_geo: Path, target_geo: Path
) -> pd.DataFrame:
    """Return per-civic-association tests and derived download/upload Mbps."""
    result = redistribute_direct(
        source_df=counts_long,
        source_geo=source_geo,
        target_geos={"civic_association": target_geo},
        count_cols=["tests", "d_product", "u_product"],
        source_id="geoid",
    )
    wide = result.pivot_table(
        index="geoid", columns="measure", values="value", aggfunc="first"
    ).reset_index()
    wide.columns.name = None
    wide = wide.rename(
        columns={
            "tests_direct": "tests",
            "d_product_direct": "d_product",
            "u_product_direct": "u_product",
        }
```

```
    )
    valid = wide["tests"] > 0
    wide["download_mbps"] = (wide["d_product"] / wide["tests"].where(valid)) / 1000
    wide["upload_mbps"] = (wide["u_product"] / wide["tests"].where(valid)) / 1000
    return wide[["geoid", "tests", "download_mbps", "upload_mbps"]]


def build_counts_long(tiles: gpd.GeoDataFrame) -> pd.DataFrame:
    """Build the long-format count frame from raw Ookla tiles."""
    t = tiles[tiles["tests"] >= MIN_TESTS].copy()
    t["d_product"] = t["avg_d_kbps"] * t["tests"]
    t["u_product"] = t["avg_u_kbps"] * t["tests"]
    long = t[["quadkey", "tests", "d_product", "u_product"]].melt(
        id_vars="quadkey", var_name="measure", value_name="value"
    )
    long = long.rename(columns={"quadkey": "geoid"})
    long["year"] = config.OOKLA_YEAR
    return long


def main() -> None:
    tiles = gpd.read_file(config.OOKLA_TILES)
    tiles["geoid"] = tiles["quadkey"].astype(str)
    long = build_counts_long(tiles)
    # Write a geoid-keyed source file for redistribute_direct
    src = tiles[["geoid", "geometry"]].copy()
    src_path = config.DATA / "_ookla_src.geojson"
    src.to_file(src_path, driver="GeoJSON")
    out = redistribute_broadband(long, src_path, config.CIVIC_ASSOC)
    out.to_csv(config.CIVIC_BROADBAND, index=False)
    src_path.unlink(missing_ok=True)
    print(f"Wrote {len(out)} civic-association broadband rows → {config.CIVIC_BROADBAND}")


if __name__ == "__main__":
```

```
    main()
```

10.7 Combine and derive metrics

Join the redistributed income and broadband measures onto the civic association polygons and compute the income-to-speed ratio and the bivariate income×speed classes.

`pipeline/combine.py`

```
"""Combine income + broadband and compute derived policy metrics."""
from __future__ import annotations

import geopandas as gpd
import pandas as pd

from pipeline import config


def _tercile(series: pd.Series) -> pd.Series:
    """Return 1/2/3 tercile labels by rank (robust to ties and NaN)."""
    ranks = series.rank(method="first")
    return pd.qcut(ranks, 3, labels=[1, 2, 3]).astype("Int64")


def add_derived_metrics(df: pd.DataFrame) -> pd.DataFrame:
    """Add income_speed_ratio and bivariate_class (income-speed terciles)."""
    out = df.copy()
    out["income_speed_ratio"] = out["mean_income"] / out["download_mbps"].where(
        out["download_mbps"] > 0
    )
    inc = _tercile(out["mean_income"])
    spd = _tercile(out["download_mbps"])
    out["bivariate_class"] = inc.astype(str) + "-" + spd.astype(str)
    return out
```

```
def main() -> None:
    civ = gpd.read_file(config.CIVIC_ASSOC)[["geoid", "region_name", "geometry"]]
    income = pd.read_csv(config.CIVIC_INCOME, dtype={"geoid": str})
    broadband = pd.read_csv(config.CIVIC_BROADBAND, dtype={"geoid": str})
    merged = civ.merge(income, on="geoid", how="left").merge(
        broadband, on="geoid", how="left"
    )
    merged = add_derived_metrics(merged)
    gdf = gpd.GeoDataFrame(merged, geometry="geometry", crs=civ.crs)
    gdf.to_file(config.CIVIC_COMBINED, driver="GeoJSON")
    print(f"Wrote {len(gdf)} combined civic-association rows → {config.CIVIC_COMBINED}")


if __name__ == "__main__":
    main()
```

10.8 Acquire parcels

Page through the Arlington MHUD FeatureServer and reduce parcel polygons to unit-bearing centroids.

`pipeline/acquire_parcels.py`

```
"""Acquire Arlington MHUD parcel polygons and reduce them to unit-bearing centroids."""
from __future__ import annotations

import json
import urllib.parse
import urllib.request

import geopandas as gpd
import pandas as pd
```

```
from pipeline import config

LAYER = (
    "https://arlgis.arlingtonva.us/arcgis/rest/services/Open_Data/"
    "od_MHUD_Polygons/FeatureServer/0/query"
)
PAGE = 2000


def _fetch_page(offset: int) -> gpd.GeoDataFrame | None:
    params = {
        "where": "1=1",
        "outFields": "Total_Units,Unit_Type",
        "outSR": "4326",
        "resultOffset": str(offset),
        "resultRecordCount": str(PAGE),
        "f": "geojson",
    }
    url = LAYER + "?" + urllib.parse.urlencode(params)
    with urllib.request.urlopen(url, timeout=60) as resp:  # noqa: S310
        data = json.load(resp)
    feats = data.get("features", [])
    if not feats:
        return None
    return gpd.GeoDataFrame.from_features(feats, crs="EPSG:4326")


def fetch_parcels() -> gpd.GeoDataFrame:
    """Page through the MHUD FeatureServer and return all parcel polygons."""
    parts: list[gpd.GeoDataFrame] = []
    offset = 0
    while True:
        page = _fetch_page(offset)
        if page is None or len(page) == 0:
            break
```

```
        parts.append(page)
        if len(page) < PAGE:
            break
        offset += PAGE
    gdf = gpd.GeoDataFrame(pd.concat(parts, ignore_index=True), crs="EPSG:4326")
    return gdf


def to_centroids(polys: gpd.GeoDataFrame) -> gpd.GeoDataFrame:
    """Reduce parcel polygons to centroids, keeping Total_Units and Unit_Type."""
    cent = polys.copy()
    # representative_point() is guaranteed inside the polygon (unlike centroid)
    cent["geometry"] = cent.geometry.representative_point()
    return cent[["Total_Units", "Unit_Type", "geometry"]]


def main() -> None:
    polys = fetch_parcels()
    pts = to_centroids(polys)
    pts.to_file(config.PARCELS, driver="GeoJSON")
    print(f"Wrote {len(pts)} parcel centroids → {config.PARCELS}")


if __name__ == "__main__":
    main()
```

10.9 Redistribute income through parcels

Explode parcels into per-unit points and redistribute block-group income through them (the dasymetric second approach).

`pipeline/redistribute_income_parcels.py`

```
"""Redistribute income through unit-weighted parcels (dasymetric), derive mean income."""
from __future__ import annotations

from pathlib import Path

import geopandas as gpd
import pandas as pd
from sdc_redistribute import redistribute_parcels

from pipeline import config


def explode_by_units(parcels: gpd.GeoDataFrame) -> gpd.GeoDataFrame:
    """Replicate each parcel into Total_Units identical points; drop units ≤ 0."""
    p = parcels[parcels["Total_Units"] > 0].copy()
    reps = p["Total_Units"].astype(int).to_numpy()
    out = p.loc[p.index.repeat(reps)].reset_index(drop=True)
    return out


def redistribute_income_parcels(
    counts_long: pd.DataFrame,
    parcels: gpd.GeoDataFrame,
    source_geo: Path,
    target_geo: Path,
) -> pd.DataFrame:
    """Return per-civic-association agg_income, households, derived mean_income."""
    points = explode_by_units(parcels)
    result = redistribute_parcels(
        source_df=counts_long,
        parcel_centroids=points,
        source_geo=source_geo,
        target_geos={"civic_association": target_geo},
        count_cols=["agg_income", "households"],
        source_id="geoid",
```

```
    )
    wide = result.pivot_table(
        index="geoid", columns="measure", values="value", aggfunc="first"
    ).reset_index()
    wide.columns.name = None
    wide = wide.rename(
        columns={"agg_income_parcels": "agg_income", "households_parcels": "households"}
    )
    wide["mean_income"] = wide["agg_income"] / wide["households"].where(
        wide["households"] > 0
    )
    return wide[["geoid", "agg_income", "households", "mean_income"]]


def main() -> None:
    parcels = gpd.read_file(config.PARCELS)
    bg = gpd.read_file(config.ACS_COUNTS).rename(columns={"GEOID": "geoid"})
    long = bg[["geoid", "agg_income", "households"]].melt(
        id_vars="geoid", var_name="measure", value_name="value"
    )
    long["year"] = config.ACS_YEAR

    src_path = config.DATA / "_acs_src_parcels.geojson"
    bg[["geoid", "geometry"]].to_file(src_path, driver="GeoJSON")
    out = redistribute_income_parcels(long, parcels, src_path, config.CIVIC_ASSOC)
    src_path.unlink(missing_ok=True)

    out.to_csv(config.CIVIC_INCOME_PARCELS, index=False)
    print(f"Wrote {len(out)} parcel-weighted income rows → {config.CIVIC_INCOME_PARCELS}")


if __name__ == "__main__":
    main()
```

10.10 Compare methods

Join the area-weighted and parcel-weighted income and quantify the difference per civic association.

`pipeline/compare_methods.py`

```
"""Compare area-weighted vs parcel-weighted income per civic association."""
from __future__ import annotations

import geopandas as gpd
import pandas as pd

from pipeline import config


def compare_income(area: pd.DataFrame, parcel: pd.DataFrame) -> pd.DataFrame:
    """Join the two methods' mean income; add diff (parcel - area) and pct_diff."""
    a = area[["geoid", "mean_income"]].rename(columns={"mean_income": "mean_income_area"})
    p = parcel[["geoid", "mean_income"]].rename(columns={"mean_income": "mean_income_parcel"})
    out = a.merge(p, on="geoid", how="outer")
    out["diff"] = out["mean_income_parcel"] - out["mean_income_area"]
    out["pct_diff"] = 100 * out["diff"] / out["mean_income_area"].where(
        out["mean_income_area"] > 0
    )
    return out


def main() -> None:
    civ = gpd.read_file(config.CIVIC_ASSOC)[["geoid", "region_name", "geometry"]]
    area = pd.read_csv(config.CIVIC_INCOME, dtype={"geoid": str})
    parcel = pd.read_csv(config.CIVIC_INCOME_PARCELS, dtype={"geoid": str})
    cmp = compare_income(area, parcel)
    gdf = civ.merge(cmp, on="geoid", how="left")
    gdf = gpd.GeoDataFrame(gdf, geometry="geometry", crs=civ.crs)
```

```
    # Honest accounting: report parcel-method household total vs ACS source.
    acs = gpd.read_file(config.ACS_COUNTS)
    src_hh = float(acs["households"].sum())
    parcel_hh = float(pd.read_csv(config.CIVIC_INCOME_PARCELS)["households"].sum())
    pct = 100 * (parcel_hh - src_hh) / src_hh if src_hh else 0.0
    print(f"Household totals, ACS source: {src_hh:,.0f}; parcel method: {parcel_hh:,.0f} ({pct

    gdf.to_file(config.CIVIC_INCOME_COMPARISON, driver="GeoJSON")
    print(f"Wrote {len(gdf)} comparison rows → {config.CIVIC_INCOME_COMPARISON}")


if __name__ == "__main__":
    main()
```

10.11 Validate

Confirm household totals are preserved by the redistribution and that no speeds or incomes are negative.

`pipeline/validate.py`

```
"""Integrity checks on redistributed outputs."""
from __future__ import annotations

import pandas as pd


def check_total_preserved(source_total: float, target_total: float, tol: float = 0.01) -> None
    """Assert target total matches source total within relative tolerance ``tol``."""
    if source_total == 0:
        return
    rel = abs(target_total - source_total) / abs(source_total)
    assert rel <= tol, f"total drift {rel:.3%} exceeds {tol:.3%}"
```

```
def check_ranges(df: pd.DataFrame) -> None:
    """Assert speed/income columns are non-negative where present."""
    for col in ("download_mbps", "upload_mbps", "mean_income", "households"):
        if col in df.columns:
            vals = df[col].dropna()
            assert (vals >= 0).all(), f"{col} has negative values"


def main() -> None:
    import geopandas as gpd
    from pipeline import config

    combined = gpd.read_file(config.CIVIC_COMBINED)
    acs = gpd.read_file(config.ACS_COUNTS)
    check_total_preserved(
        acs["households"].sum(), combined["households"].sum(), tol=0.02
    )
    check_ranges(combined)
    print("Validation passed.")


if __name__ == "__main__":
    main()
```

10.12 Run the full pipeline

Wire every stage into one command: `uv run python -m pipeline.run`.

`pipeline/run.py`

```
"""Run the full pipeline end to end (acquisition → outputs → validation)."""
from __future__ import annotations

from pipeline import (
```

```
    acquire_acs,
    acquire_geographies,
    acquire_ookla,
    acquire_parcels,
    combine,
    compare_methods,
    redistribute_broadband,
    redistribute_income,
    redistribute_income_parcels,
    validate,
)


def main() -> None:
    acquire_geographies.main()
    acquire_acs.main()
    acquire_ookla.main()
    acquire_parcels.main()
    redistribute_income.main()
    redistribute_broadband.main()
    combine.main()
    redistribute_income_parcels.main()
    compare_methods.main()
    validate.main()
    print("Pipeline complete.")


if __name__ == "__main__":
    main()
```

11 The Complete Pipeline Code (R)

This appendix reproduces the runnable R counterpart to the worked example. It is generated directly from the `pipeline-r/` directory in the companion repository. The R pipeline reads the same intermediate data the Python pipeline produces and performs the redistribution, combination, and validation in R, built on the `sdc.redistribute` package:

```
# install.packages("pak"); pak::pak("dads2busy/sdc.redistribute")
# from the repository root:
# Rscript pipeline-r/run.R
```

Data acquisition is shown as a recipe (`acquire-recipe.R`) rather than re-run. The `compare_to_python.R` module confirms the R results match the Python results.

11.1 Configuration

Shared paths, the projected CRS used for area weighting, and the count constants. Mirrors the Python configuration.

`pipeline-r/config.R`

```
# Shared configuration for the R pipeline. Mirrors pipeline/config.py.
# Run all modules from the repository root (e.g. `Rscript pipeline-r/run.R`).

PROJECT_CRS <- 3857   # match the CRS sdc-redistribute uses internally in Python
ACS_YEAR    <- 2021
OOKLA_YEAR  <- 2023
MIN_TESTS   <- 5      # statistical-reliability filter on source tiles
```

```
DATA <- "data"
path_data <- function(...) file.path(DATA, ...)

# Inputs (shared with the Python pipeline)
CIVIC_ASSOC <- path_data("va013_geo_arl_2021_civic_associations.geojson")
ACS_COUNTS  <- path_data("va013_acs_income_counts.geojson")
OOKLA_TILES <- path_data("ookla_tiles_arlington.geojson")
PARCELS     <- path_data("va013_parcels.geojson")

# R outputs (suffixed _r so they never clobber the Python outputs)
CIVIC_INCOME_R            <- path_data("civic_income_r.csv")
CIVIC_BROADBAND_R         <- path_data("civic_broadband_r.csv")
CIVIC_COMBINED_R          <- path_data("civic_combined_r.geojson")
CIVIC_INCOME_PARCELS_R    <- path_data("civic_income_parcels_r.csv")
CIVIC_INCOME_COMPARISON_R <- path_data("civic_income_comparison_r.geojson")

# Python outputs (for parity comparison)
CIVIC_COMBINED_PY <- path_data("civic_combined.geojson")
```

11.2 Redistribute income

Area-weight aggregate income and households onto civic associations with `sdc.redistribute::redistribute_` then derive mean income.

`pipeline-r/redistribute_income.R`

```
# Area-weight ACS income counts onto civic associations; derive mean income.
# Parallels pipeline/redistribute_income.py.
source("pipeline-r/config.R")
suppressPackageStartupMessages({
  library(sf)
  library(sdc.redistribute)
})
```

```r
redistribute_income <- function(acs_path = ACS_COUNTS, civic_path = CIVIC_ASSOC) {
  bg <- sf::st_read(acs_path, quiet = TRUE)
  names(bg)[names(bg) == "GEOID"] <- "geoid"
  civ <- sf::st_read(civic_path, quiet = TRUE)

  bg  <- sf::st_transform(bg,  PROJECT_CRS)
  civ <- sf::st_transform(civ, PROJECT_CRS)

  # agg_income and households are extensive counts; redistribute both, then
  # derive mean income (an intensive quantity) as their ratio.
  out <- redistribute_direct(bg, civ, extensive = c("agg_income", "households"))
  out$mean_income <- ifelse(out$households > 0, out$agg_income / out$households, NA_real_)

  d <- sf::st_drop_geometry(out)
  d[, c("geoid", "agg_income", "households", "mean_income")]
}

run_income <- function() {
  out <- redistribute_income()
  write.csv(out, CIVIC_INCOME_R, row.names = FALSE)
  cat(sprintf("Wrote %d civic-association income rows → %s\n", nrow(out), CIVIC_INCOME_R))
}

if (sys.nframe() == 0) run_income()
```

11.3 Redistribute broadband

Rebuild the speed×tests product, redistribute it with the test counts, then derive the count-weighted mean download and upload speeds.

`pipeline-r/redistribute_broadband.R`

```r
# Area-weight Ookla broadband onto civic associations; derive count-weighted
# mean speeds. Parallels pipeline/redistribute_broadband.py.
```

```r
source("pipeline-r/config.R")
suppressPackageStartupMessages({
  library(sf)
  library(sdc.redistribute)
})

redistribute_broadband <- function(tiles_path = OOKLA_TILES, civic_path = CIVIC_ASSOC) {
  tiles <- sf::st_read(tiles_path, quiet = TRUE)
  tiles <- tiles[tiles$tests >= MIN_TESTS, ]
  # Speed is intensive: rebuild the extensive speed x tests product so the
  # count-weighted mean can be recovered after redistribution.
  tiles$d_product <- tiles$avg_d_kbps * tiles$tests
  tiles$u_product <- tiles$avg_u_kbps * tiles$tests

  civ <- sf::st_read(civic_path, quiet = TRUE)
  tiles <- sf::st_transform(tiles, PROJECT_CRS)
  civ   <- sf::st_transform(civ,   PROJECT_CRS)

  out <- redistribute_direct(tiles, civ, extensive = c("tests", "d_product", "u_product"))
  valid <- out$tests > 0
  out$download_mbps <- ifelse(valid, (out$d_product / out$tests) / 1000, NA_real_)
  out$upload_mbps   <- ifelse(valid, (out$u_product / out$tests) / 1000, NA_real_)

  d <- sf::st_drop_geometry(out)
  d[, c("geoid", "tests", "download_mbps", "upload_mbps")]
}

run_broadband <- function() {
  out <- redistribute_broadband()
  write.csv(out, CIVIC_BROADBAND_R, row.names = FALSE)
  cat(sprintf("Wrote %d civic-association broadband rows → %s\n", nrow(out), CIVIC_BROADBAND_
}

if (sys.nframe() == 0) run_broadband()
```

11.4 Combine and derive metrics

Join redistributed income and broadband onto the civic polygons and compute the income-to-speed ratio and bivariate income×speed classes.

`pipeline-r/combine.R`

```
# Join redistributed income + broadband onto civic polygons; derive metrics.
# Parallels pipeline/combine.py.
source("pipeline-r/config.R")
suppressPackageStartupMessages(library(sf))

tercile <- function(x) {
  # 1/2/3 labels by rank (robust to ties and NA), mirroring pandas qcut on ranks.
  r <- rank(x, ties.method = "first", na.last = "keep")
  br <- stats::quantile(r, probs = c(0, 1/3, 2/3, 1), na.rm = TRUE)
  cut(r, breaks = br, labels = c("1", "2", "3"), include.lowest = TRUE)
}

add_derived_metrics <- function(df) {
  df$income_speed_ratio <- ifelse(df$download_mbps > 0,
                                  df$mean_income / df$download_mbps, NA_real_)
  inc <- tercile(df$mean_income)
  spd <- tercile(df$download_mbps)
  df$bivariate_class <- paste0(as.character(inc), "-", as.character(spd))
  df
}

run_combine <- function() {
  civ <- sf::st_read(CIVIC_ASSOC, quiet = TRUE)[, c("geoid", "region_name")]
  income    <- read.csv(CIVIC_INCOME_R,    colClasses = c(geoid = "character"))
  broadband <- read.csv(CIVIC_BROADBAND_R, colClasses = c(geoid = "character"))
  civ$geoid <- as.character(civ$geoid)

  merged <- merge(civ, income, by = "geoid", all.x = TRUE)
  merged <- merge(merged, broadband, by = "geoid", all.x = TRUE)
```

```
  merged <- add_derived_metrics(merged)

  sf::st_write(merged, CIVIC_COMBINED_R, delete_dsn = TRUE, quiet = TRUE)
  cat(sprintf("Wrote %d combined civic-association rows → %s\n",
              nrow(merged), CIVIC_COMBINED_R))
}

if (sys.nframe() == 0) run_combine()
```

11.5 Redistribute income through parcels

Dasymetric income via `redistribute_parcels()`, weighting parcels by their unit count (the second approach).

`pipeline-r/redistribute_income_parcels.R`

```
# Dasymetric income redistribution through unit-weighted parcels.
# Parallels pipeline/redistribute_income_parcels.py: Python replicates each
# parcel into Total_Units identical points; here we pass Total_Units as the
# per-point weight, which splits each source value in the same proportion.
source("pipeline-r/config.R")
suppressPackageStartupMessages({
  library(sf)
  library(sdc.redistribute)
})

redistribute_income_parcels <- function(acs_path = ACS_COUNTS,
                                        parcels_path = PARCELS,
                                        civic_path = CIVIC_ASSOC) {
  bg <- sf::st_read(acs_path, quiet = TRUE)
  names(bg)[names(bg) == "GEOID"] <- "geoid"
  parcels <- sf::st_read(parcels_path, quiet = TRUE)
  parcels <- parcels[parcels$Total_Units > 0, ]
  civ <- sf::st_read(civic_path, quiet = TRUE)
```

```
  bg      <- sf::st_transform(bg,      PROJECT_CRS)
  parcels <- sf::st_transform(parcels, PROJECT_CRS)
  civ     <- sf::st_transform(civ,     PROJECT_CRS)

  out <- redistribute_parcels(bg, civ, parcels,
                              extensive = c("agg_income", "households"),
                              weights = "Total_Units")
  out$mean_income <- ifelse(out$households > 0, out$agg_income / out$households, NA_real_)

  d <- sf::st_drop_geometry(out)
  d[, c("geoid", "agg_income", "households", "mean_income")]
}

run_income_parcels <- function() {
  out <- redistribute_income_parcels()
  write.csv(out, CIVIC_INCOME_PARCELS_R, row.names = FALSE)
  cat(sprintf("Wrote %d parcel-weighted income rows → %s\n",
              nrow(out), CIVIC_INCOME_PARCELS_R))
}

if (sys.nframe() == 0) run_income_parcels()
```

11.6 Compare methods

Join the area-weighted and parcel-weighted income and quantify the difference per civic association.

`pipeline-r/compare_methods.R`

```
# Compare area-weighted vs parcel-weighted income per civic association.
# Parallels pipeline/compare_methods.py.
source("pipeline-r/config.R")
suppressPackageStartupMessages(library(sf))
```

```
compare_income <- function(area, parcel) {
  a <- data.frame(geoid = as.character(area$geoid),
                  mean_income_area = area$mean_income, stringsAsFactors = FALSE)
  p <- data.frame(geoid = as.character(parcel$geoid),
                  mean_income_parcel = parcel$mean_income, stringsAsFactors = FALSE)
  out <- merge(a, p, by = "geoid", all = TRUE)
  out$diff <- out$mean_income_parcel - out$mean_income_area
  out$pct_diff <- ifelse(out$mean_income_area > 0,
                         100 * out$diff / out$mean_income_area, NA_real_)
  out
}

run_compare <- function() {
  civ    <- sf::st_read(CIVIC_ASSOC, quiet = TRUE)[, c("geoid", "region_name")]
  civ$geoid <- as.character(civ$geoid)
  area   <- read.csv(CIVIC_INCOME_R,         colClasses = c(geoid = "character"))
  parcel <- read.csv(CIVIC_INCOME_PARCELS_R, colClasses = c(geoid = "character"))
  cmp <- compare_income(area, parcel)
  out <- merge(civ, cmp, by = "geoid", all.x = TRUE)
  sf::st_write(out, CIVIC_INCOME_COMPARISON_R, delete_dsn = TRUE, quiet = TRUE)
  cat(sprintf("Wrote %d comparison rows → %s\n", nrow(out), CIVIC_INCOME_COMPARISON_R))
}

if (sys.nframe() == 0) run_compare()
```

11.7 Validate

Confirm household totals are preserved and that no speeds or incomes are negative.

`pipeline-r/validate.R`

```
# Integrity checks on redistributed outputs. Parallels pipeline/validate.py.
source("pipeline-r/config.R")
suppressPackageStartupMessages(library(sf))
```

```r
check_total_preserved <- function(source_total, target_total, tol = 0.01) {
  if (source_total == 0) return(invisible(TRUE))
  rel <- abs(target_total - source_total) / abs(source_total)
  if (rel > tol) {
    stop(sprintf("total drift %.3f%% exceeds %.3f%%", 100 * rel, 100 * tol), call. = FALSE)
  }
  invisible(TRUE)
}

check_ranges <- function(df) {
  for (col in c("download_mbps", "upload_mbps", "mean_income", "households")) {
    if (col %in% names(df)) {
      vals <- df[[col]][!is.na(df[[col]])]
      if (any(vals < 0)) stop(sprintf("%s has negative values", col), call. = FALSE)
    }
  }
  invisible(TRUE)
}

run_validate <- function() {
  combined <- sf::st_drop_geometry(sf::st_read(CIVIC_COMBINED_R, quiet = TRUE))
  acs <- sf::st_drop_geometry(sf::st_read(ACS_COUNTS, quiet = TRUE))
  check_total_preserved(sum(acs$households), sum(combined$households, na.rm = TRUE), tol = 0.0
  check_ranges(combined)
  cat("Validation passed.\n")
}

if (sys.nframe() == 0) run_validate()
```

11.8 Run the R pipeline

Run the redistribute/combine/validate stages end to end: `Rscript pipeline-r/run.R`.

pipeline-r/run.R

```
# Run the runnable R stages end to end (acquisition is a recipe; see
# pipeline-r/acquire-recipe.R). Parallels pipeline/run.py.
# Usage from the repository root: Rscript pipeline-r/run.R
source("pipeline-r/redistribute_income.R")
source("pipeline-r/redistribute_broadband.R")
source("pipeline-r/combine.R")
source("pipeline-r/redistribute_income_parcels.R")
source("pipeline-r/compare_methods.R")
source("pipeline-r/validate.R")

run_income()
run_broadband()
run_combine()
run_income_parcels()
run_compare()
run_validate()
cat("R pipeline complete.\n")
```

11.9 Acquisition recipe

How the same inputs would be produced in R (`tidycensus`/`tigris`/`ooklaOpenDataR`). Shown for reference; not executed by the pipeline.

pipeline-r/acquire-recipe.R

```
# Data-acquisition recipe (NOT executed by run.R). Shows how the same
# intermediate inputs the R pipeline reads would be produced in R, paralleling
# the Python acquire_*.py modules. Requires a Census API key and network access.
#
# install.packages(c("tidycensus", "tigris", "sf"))
# # ooklaOpenDataR is on GitHub: remotes::install_github("teamookla/ooklaOpenDataR")

if (FALSE) {  # recipe guard: never runs
```

```
  library(tidycensus)
  library(tigris)
  library(sf)

  # 1. Census block-group geometries for Arlington County, VA (FIPS 51013),
  #    2021 vintage. Parallels acquire_geographies.py.
  bg_geo <- tigris::block_groups(state = "51", county = "013", year = 2021, cb = FALSE)

  # 2. ACS extensive counts: aggregate household income (B19025_001) and
  #    households (B11001_001). Parallels acquire_acs.py. NEVER fetch the median.
  acs <- tidycensus::get_acs(
    geography = "block group", state = "51", county = "013",
    year = 2021, survey = "acs5", geometry = TRUE,
    variables = c(agg_income = "B19025_001", households = "B11001_001"),
    output = "wide"
  )
  # acs has agg_incomeE / householdsE columns; rename to agg_income / households.

  # 3. Ookla fixed-broadband tiles, clipped to Arlington. Parallels acquire_ookla.py.
  #    ooklaOpenDataR::get_performance_tiles(service = "fixed", year = 2023,
  #      quarter = 2) returns the global tile set; clip to the county bbox.

  # 4. Parcels: county MHUD parcel points carrying Total_Units. Parallels
  #    acquire_parcels.py (an ArcGIS FeatureServer; read via arcgislayers or sf).
}
```

11.10 Parity with the Python pipeline

Confirm the R outputs match the Python pipeline's mean income and download speed within tolerance.

`pipeline-r/compare_to_python.R`

```r
# Parity check: confirm the R combined output matches the Python combined output
# (mean_income and download_mbps) within tolerance. Small differences are
# expected from GEOS/sf vs shapely/geopandas geometry handling.
source("pipeline-r/config.R")
suppressPackageStartupMessages(library(sf))

compare_to_python <- function(tol = 0.02) {
  r  <- sf::st_drop_geometry(sf::st_read(CIVIC_COMBINED_R, quiet = TRUE))
  py <- sf::st_drop_geometry(sf::st_read(CIVIC_COMBINED_PY, quiet = TRUE))
  r$geoid  <- as.character(r$geoid)
  py$geoid <- as.character(py$geoid)
  m <- merge(r, py, by = "geoid", suffixes = c("_r", "_py"))

  rel <- function(a, b) {
    ok <- is.finite(a) & is.finite(b) & b != 0
    max(abs(a[ok] - b[ok]) / abs(b[ok]))
  }
  d_income <- rel(m$mean_income_r,   m$mean_income_py)
  d_speed  <- rel(m$download_mbps_r, m$download_mbps_py)

  cat(sprintf("Parity vs Python (n=%d): max rel diff  mean_income=%.4f%%  download_mbps=%.4f%%
              nrow(m), 100 * d_income, 100 * d_speed))
  if (d_income > tol || d_speed > tol) {
    stop(sprintf("parity exceeds tolerance %.2f%%", 100 * tol), call. = FALSE)
  }
  cat(sprintf("Parity OK (within %.2f%%).\n", 100 * tol))
  invisible(TRUE)
}

if (sys.nframe() == 0) compare_to_python()
```

References

[1] "Stacks.cdc.gov." Accessed: Jun. 04, 2026. [Online]. Available: https://stacks.cdc.gov/view/cdc/59750/cdc_59750_DS1.pdf

[2] "Github.com." Accessed: Jun. 04, 2026. [Online]. Available: https://github.com/CDCgov/EPHTracking-Subcounty

[3] "results4america.org." Accessed: Jun. 04, 2026. [Online]. Available: https://results4america.org/wp-content/uploads/2021/06/Deloitte-WWC-Data-Gap-Report_vFinal-063021.pdf

[4] "Iecam.illinois.edu." Accessed: Jun. 04, 2026. [Online]. Available: https://iecam.illinois.edu/browse/subcounty-data-cautions-and-recommendations

[5] "Atlas.co." Accessed: Jun. 04, 2026. [Online]. Available: https://atlas.co/blog/modifiable-areal-unit-problem-maup/

[6] "Gisgeography.com." Accessed: Jun. 04, 2026. [Online]. Available: https://gisgeography.com/maup-modifiable-areal-unit-problem/

[7] "Speedtest by ookla global fixed and mobile network performance ..." Accessed: Jun. 04, 2026. [Online]. Available: https://github.com/teamookla/ookla-open-data

[8] "Announcing ookla open datasets." Accessed: Jun. 04, 2026. [Online]. Available: https://www.ookla.com/articles/announcing-ookla-open-datasets

[9] "American community survey 5-year data (2009-2023)." Accessed: Jun. 04, 2026. [Online]. Available: https://www.census.gov/data/developers/data-sets/acs-5year.html

[10] "American community survey data - u.s. Census bureau." Accessed: Jun. 04, 2026. [Online]. Available: https://www.census.gov/programs-surveys/acs/data.html

[11] "Arlington county civic federation: home." Accessed: Jun. 04, 2026. [Online]. Available: https://www.civfed.org

[12] "Civic & citizen associations - arlington county." Accessed: Jun. 04, 2026. [Online]. Available: https://www.arlingtonva.us/Government/Topics/Community/Civic-and-Citizen-Association

[13] "Civic association polygons | arlington county, virginia. GIS open data." Accessed: Jun. 04, 2026. [Online]. Available: https://gisdata-arlgis.opendata.arcgis.com/datasets/ArlGIS::civic-association-polygons/explore?location=38.880938%2C-77.109550%2C13.55

[14] "What is areal interpolation?—ArcGIS pro | documentation." Accessed: Jun. 04, 2026. [Online]. Available: https://pro.arcgis.com/en/pro-app/latest/help/analysis/geostatistical-analyst/what-is-areal-interpolation.htm

[15] "Areal interpolation | vincent thorne." Accessed: Jun. 04, 2026. [Online]. Available: https://vinceth.net/2021/06/18/areal-interpolation.html

[16] "Spatial interpolation using areal features: A review of methods and ..." Accessed: Jun. 04, 2026. [Online]. Available: https://compass.onlinelibrary.wiley.com/doi/10.1111/gec3.12465

[17] "Arlington county, virginia - u.s. Census bureau QuickFacts." Accessed: Jun. 04, 2026. [Online]. Available: https://www.census.gov/quickfacts/fact/table/arlingtoncountyvirginia/PST045224

[18] "Mapping the digital divide: What predicts internet access across ..." Accessed: Jun. 04, 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10961931/

[19] "Bridging the digital divide: A path to universal broadband access." Accessed: Jun. 04, 2026. [Online]. Available: https://broadbandbreakfast.com/bridging-the-digital-divide-a-path-to-universal-broadband-access/

[20] "How policymakers can help bridge the digital divide in 2021 - AAF." Accessed: Jun. 04, 2026. [Online]. Available: https://www.americanactionforum.org/insight/how-policymakers-can-help-bridge-the-digital-divide-in-2021/

[21] "Areal interpolation | by ITC, university of twente." Accessed: Jun. 04, 2026. [Online]. Available: https://gistbok-ltb.ucgis.org/27/concept/7992

[22] "Speedtest® methodology | ookla®." Accessed: Jun. 04, 2026. [Online]. Available: https://www.ookla.com/resources/guides/speedtest-methodology

[23] "American community survey (ACS) - arlington county." Accessed: Jun. 04, 2026. [Online]. Available: https://www.arlingtonva.us/Government/Projects/Data-Research/Demographics/American-Community-Survey